

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



LAB 2

KHAI THÁC TẬP PHỔ BIẾN BẰNG THUẬT TOÁN PREPOST

Mã số sinh viên Họ và tên

22120187 TRẦN THIÊN LỘC

23120095 LƯU ĐỨC TOÀN

23120245 NGUYỄN QUANG DUY

23120258 LƯU TRỌNG HIẾU

23120260 VĂN ĐÌNH HIẾU

Giảng viên hướng dẫn GS.TS Lê Hoài Bắc
Th.S Lê Nhật Nam

Môn học Khai thác dữ liệu và ứng dụng

Thành phố Hồ Chí Minh – 2026

Lời cảm ơn

Nhóm chúng em xin gửi lời cảm ơn chân thành đến quý thầy cô phụ trách học phần *Khai thác dữ liệu và ứng dụng* đã hướng dẫn, cung cấp kiến thức nền tảng và định hướng thực hiện đồ án.

Trong quá trình thực hiện, nhóm đã có cơ hội tìm hiểu sâu hơn về bài toán khai thác tập phổ biến, các cấu trúc dữ liệu đặc trưng và phương pháp đánh giá hiệu năng thuật toán trên các tập dữ liệu benchmark. Những kiến thức này giúp nhóm hiểu rõ hơn vai trò của Frequent Itemset Mining trong khai phá dữ liệu và các ứng dụng thực tế như phân tích giỏ hàng.

Do giới hạn về thời gian và kinh nghiệm, báo cáo có thể còn thiếu sót. Nhóm mong nhận được góp ý từ quý thầy cô để hoàn thiện hơn.

Tóm tắt

Khai thác tập phổ biến là một bài toán nền tảng trong khai phá dữ liệu, được sử dụng rộng rãi trong sinh luật kết hợp, phân tích giỏ hàng, phát hiện mẫu trong log hệ thống và nhiều lĩnh vực khác. Đề án này tập trung nghiên cứu và cài đặt thuật toán **PrePost**, một thuật toán khai thác tập phổ biến dựa trên cấu trúc **PPC-tree** và **N-list**.

Khác với các thuật toán sinh ứng viên như Apriori, PrePost mã hóa quan hệ tổ tiên – hậu duệ giữa các node trên cây bằng cặp thứ tự duyệt trước và duyệt sau. Từ đó, thuật toán có thể tính support của các itemset thông qua thao tác trên N-list mà không cần quét lại toàn bộ cơ sở dữ liệu giao dịch nhiều lần.

Trong đề án, nhóm trình bày cơ sở lý thuyết của bài toán Frequent Itemset Mining, phân tích chi tiết thuật toán PrePost, minh họa thuật toán bằng ví dụ thủ công, cài đặt thuật toán bằng ngôn ngữ Julia, kiểm tra độ đúng bằng cách so sánh với SPMF và đánh giá hiệu năng trên các tập dữ liệu benchmark. Ngoài ra, nhóm áp dụng kết quả frequent itemset vào bài toán phân tích giỏ hàng thông qua việc sinh luật kết hợp.

Từ khóa: Frequent Itemset Mining, PrePost, PPC-tree, N-list, association rules, market basket analysis.

Danh mục từ viết tắt

FIM	Frequent Itemset Mining – Khai thác tập phổ biến.
FI	Frequent Itemset – Tập phổ biến.
CFI	Closed Frequent Itemset – Tập phổ biến đóng.
MFI	Maximal Frequent Itemset – Tập phổ biến tối đại.
TDB	Transaction Database – Cơ sở dữ liệu giao dịch.
TID	Transaction Identifier – Mã định danh giao dịch.
minsup	Minimum Support – Ngưỡng hỗ trợ tối thiểu.
sup	Support – Độ hỗ trợ của một itemset.
conf	Confidence – Độ tin cậy của một luật kết hợp.
lift	Lift – Độ đo mức độ phụ thuộc giữa hai vế của luật kết hợp.
PPC-tree	Pre-order Post-order Coding Tree – Cây mã hóa theo thứ tự duyệt trước và duyệt sau.
pre	Pre-order code – Mã thứ tự duyệt trước của một node trong PPC-tree.
post	Post-order code – Mã thứ tự duyệt sau của một node trong PPC-tree.
N-list	Node-list – Danh sách mã hóa các node dùng để tính support trong PrePost.
SPMF	Sequential Pattern Mining Framework – Bộ công cụ tham chiếu cho khai phá mẫu.
CLI	Command Line Interface – Giao diện dòng lệnh.
RAM	Random Access Memory – Bộ nhớ truy cập ngẫu nhiên.
CSV	Comma-Separated Values – Định dạng dữ liệu bảng phân tách bằng dấu phẩy.
MBA	Market Basket Analysis – Phân tích giỏ hàng.
AR	Association Rule – Luật kết hợp.
EDA	Exploratory Data Analysis – Phân tích dữ liệu khám phá.
SKU	Stock Keeping Unit – Đơn vị lưu kho hoặc mã sản phẩm.
minconf	Minimum Confidence – Ngưỡng độ tin cậy tối thiểu.
lev	Leverage – Độ đo chênh lệch giữa xác suất đồng xuất hiện thực tế và kỳ vọng độc lập.
conv	Conviction – Độ đo mức độ phụ thuộc có hướng của luật kết hợp.

Danh sách hình vẽ

1	PPC-tree của ví dụ cơ sở trước khi gán mã <i>pre</i> và <i>post</i>	10
2	PPC-tree của ví dụ cơ sở sau khi gán mã <i>pre</i> và <i>post</i>	11
3	PPC-tree một nhánh trong ví dụ đặc biệt	17
4	Pipeline cài đặt thuật toán PrePost	24
5	Giải mã khai thác đệ quy bằng N-list trong phiên bản tối ưu	26
6	Pipeline cài đặt thuật toán PrePost	37
7	Giải mã khai thác đệ quy bằng N-list trong phiên bản tối ưu	38
8	Top 15 item xuất hiện nhiều nhất trong tập groceries	48
9	Ma trận đồng xuất hiện của top 15 item phổ biến	49
10	Phân bố số item trong mỗi giao dịch	50
11	Heatmap lift của top 10 luật kết hợp	54
12	Bubble chart: quan hệ giữa support, confidence và lift của top 10 luật . .	54
13	So sánh lift của top 10 luật kết hợp	55

Danh sách bảng

2	Cơ sở dữ liệu giao dịch của ví dụ cơ sở	9
3	Support của các item đơn trong ví dụ cơ sở	9
4	Cơ sở dữ liệu giao dịch sau khi sắp xếp	10
5	Các node của PPC-tree sau khi gán mã <i>pre</i> và <i>post</i>	11
6	N-list của các item đơn trong ví dụ cơ sở	12
7	Các frequent 2-itemset trong ví dụ cơ sở	13
8	Các frequent 3-itemset trong ví dụ cơ sở	14
9	Tập frequent itemset cuối cùng của ví dụ cơ sở	15
10	Kiểm tra chéo support của toàn bộ 2-itemset	15
11	Kiểm tra chéo support của toàn bộ 3-itemset	16
12	Cơ sở dữ liệu giao dịch của ví dụ một nhánh	16
13	Support của các item đơn trong ví dụ một nhánh	17
14	Cơ sở dữ liệu giao dịch sau khi sắp xếp	17
15	Mã <i>pre</i> và <i>post</i> trong PPC-tree một nhánh	18
16	N-list của các item đơn trong ví dụ một nhánh	18
17	Tập frequent itemset cuối cùng của ví dụ một nhánh	19
18	Môi trường và công cụ cài đặt	21
19	Tổ chức các file mã nguồn chính	21
20	Năm CSDL toy dùng cho kiểm thử Level 2	28
21	Kết quả kiểm tra thuật toán trên năm CSDL toy	29
22	Kết quả so sánh phiên bản cơ bản và phiên bản tối ưu	30
23	Tổng hợp mức độ đáp ứng các yêu cầu cài đặt	31
24	Môi trường và công cụ cài đặt	33
25	Tổ chức các file mã nguồn chính	34
26	Năm CSDL toy dùng cho kiểm thử Level 2	41
27	Kết quả kiểm tra thuật toán trên năm CSDL toy	41
28	Kết quả so sánh phiên bản cơ bản và phiên bản tối ưu	42
29	Tổng hợp mức độ đáp ứng các yêu cầu cài đặt	43
30	Tổng quan dữ liệu <code>groceries.txt</code> sau tiền xử lý	47
31	Top 15 item xuất hiện nhiều nhất trong dữ liệu	48
32	Thống kê kích thước giỏ hàng	50
33	Số lượng association rules theo từng cặp <code>minsup</code> và <code>minconf</code>	51
34	Cấu hình ngưỡng được chọn cho phần ứng dụng	51
35	Một số frequent itemset tiêu biểu với <code>minsup = 0.020</code>	52
36	Top 10 luật kết hợp theo <code>lift</code>	53
37	Diễn giải một số luật tiêu biểu và đề xuất hành động	56

Mục lục

Lời cảm ơn	i
Tóm tắt	ii
Danh mục từ viết tắt	iii
Danh mục hình vẽ	iv
Danh mục bảng biểu	v
1 Nền tảng lý thuyết	1
1.1 Bài toán khai thác tập mục phổ biến	1
1.2 Tính chất Apriori	2
1.3 Tổng quan thuật toán PrePost	3
1.4 Cấu trúc PPC-tree	3
1.5 Cấu trúc N-list	4
1.6 Giải mã thuật toán PrePost	5
1.7 Phân tích độ phức tạp	6
1.8 So sánh với các thuật toán FIM tiêu biểu	7
1.9 Kết luận chương	8
2 Ví dụ minh họa thuật toán PrePost	8
2.1 Ví dụ 1: Ví dụ cơ sở	8
2.1.1 Cơ sở dữ liệu giao dịch ban đầu	8
2.1.2 Bước 1: Tính support của các item đơn	9
2.1.3 Bước 2: Sắp xếp lại các giao dịch	9
2.1.4 Bước 3: Xây dựng PPC-tree	10
2.1.5 Bước 4: Gán mã pre và post	10
2.1.6 Bước 5: Tạo N-list cho các item đơn	11
2.1.7 Bước 6: Sinh frequent 2-itemset bằng N-list	12
2.1.8 Bước 7: Sinh frequent 3-itemset	13
2.1.9 Tập kết quả cuối cùng	15
2.1.10 Kiểm tra chéo bằng liệt kê thủ công	15
2.2 Ví dụ 2: Trường hợp PPC-tree một nhánh	16
2.2.1 Cơ sở dữ liệu giao dịch ban đầu	16
2.2.2 Bước 1: Tính support và xác định thứ tự toàn cục	17
2.2.3 Bước 2: Xây dựng PPC-tree	17
2.2.4 Bước 3: Gán mã pre và post	18
2.2.5 Bước 4: Tạo N-list	18

2.2.6	Bước 5: Khai thác các frequent itemset	18
2.2.7	Phân tích trường hợp PPC-tree một nhánh	19
2.3	Kết luận	20
3	Cài đặt thuật toán PrePost	20
3.1	Môi trường và công cụ	20
3.2	Tổ chức mã nguồn	21
3.3	Biểu diễn dữ liệu và cấu trúc chính	22
3.3.1	Cơ sở dữ liệu giao dịch	22
3.3.2	PPC-tree	23
3.3.3	N-list	23
3.4	Quy trình cài đặt thuật toán PrePost	24
3.5	Phiên bản cơ bản và phiên bản tối ưu	25
3.6	Xử lý đầu vào, đầu ra và giao diện dòng lệnh	26
3.7	Kiểm thử Level 1 và Level 2	27
3.7.1	Mục tiêu kiểm thử	27
3.7.2	Bộ dữ liệu kiểm thử Level 2	28
3.8	Kiểm tra Level 3: so sánh phiên bản cơ bản và tối ưu	29
3.9	Đáp ứng các mức yêu cầu cài đặt	30
3.10	Nhận xét về cài đặt	31
3.11	Khả năng tái lập	32
3.12	Kết luận chương	32
4	Cài đặt thuật toán PrePost	32
4.1	Môi trường và công cụ	33
4.2	Tổ chức mã nguồn	33
4.3	Biểu diễn dữ liệu và cấu trúc chính	35
4.3.1	Cơ sở dữ liệu giao dịch	35
4.3.2	PPC-tree	35
4.3.3	N-list	36
4.4	Quy trình cài đặt thuật toán PrePost	36
4.5	Phiên bản cơ bản và phiên bản tối ưu	37
4.6	Xử lý đầu vào, đầu ra và giao diện dòng lệnh	39
4.7	Kiểm thử Level 1 và Level 2	40
4.7.1	Mục tiêu kiểm thử	40
4.7.2	Bộ dữ liệu kiểm thử Level 2	40
4.8	Kiểm tra Level 3: so sánh phiên bản cơ bản và tối ưu	42
4.9	Đáp ứng các mức yêu cầu cài đặt	43
4.10	Nhận xét về cài đặt	44

4.11	Khả năng tái lập	44
4.12	Kết luận chương	45
5	Ứng dụng thực tế: Phân tích giỏ hàng	45
5.1	Mục tiêu ứng dụng	45
5.2	Mô tả dữ liệu và tiền xử lý	46
5.3	Khám phá dữ liệu ban đầu	47
5.3.1	Các item phổ biến nhất	47
5.3.2	Ma trận đồng xuất hiện	49
5.3.3	Phân bố kích thước giỏ hàng	49
5.4	Thiết lập khai thác frequent itemset và sinh luật kết hợp	50
5.5	Kết quả frequent itemset tiêu biểu	52
5.6	Kết quả luật kết hợp	53
5.7	Trực quan hóa luật kết hợp	53
5.8	Diễn giải kinh doanh và đề xuất hành động	55
5.9	Hạn chế của phần ứng dụng	56
5.10	Kết luận chương	57
	Tài liệu tham khảo	58
A	Phụ lục	60
A.1	Mã nguồn và notebook thực nghiệm	60
A.2	Môi trường thực nghiệm	60
A.3	Cấu trúc thư mục	61
A.4	Hướng dẫn tái lập kết quả	62
A.5	Ghi chú về kết quả sinh tự động	62

1 Nền tảng lý thuyết

1.1 Bài toán khai thác tập mục phổ biến

Khai thác tập mục phổ biến (*Frequent Itemset Mining*) là bài toán tìm các tập mục xuất hiện với tần suất không nhỏ hơn một ngưỡng cho trước trong cơ sở dữ liệu giao dịch. Đây là nền tảng của khai phá luật kết hợp và được sử dụng trong nhiều bài toán như phân tích giỏ hàng, phân tích hành vi, khai phá log và phát hiện các mẫu đồng xuất hiện [1], [2].

Cho tập tất cả các mục:

$$I = \{i_1, i_2, \dots, i_m\}. \quad (1)$$

Một *itemset* là một tập con của I . Cơ sở dữ liệu giao dịch (*transaction database*, TDB) được ký hiệu:

$$D = \{T_1, T_2, \dots, T_n\}, \quad (2)$$

trong đó mỗi giao dịch $T_j \subseteq I$ là tập các mục xuất hiện cùng nhau trong một lần quan sát và thường được gắn với một mã giao dịch (*transaction identifier*, TID).

Với itemset $X \subseteq I$, giao dịch T_j chứa X khi và chỉ khi:

$$X \subseteq T_j. \quad (3)$$

Độ hỗ trợ tuyệt đối của X trên D là số giao dịch chứa X :

$$\text{sup}_{abs}(X) = |\{T_j \in D \mid X \subseteq T_j\}|. \quad (4)$$

Độ hỗ trợ tương đối là tỉ lệ giao dịch chứa X :

$$\text{sup}_{rel}(X) = \frac{\text{sup}_{abs}(X)}{|D|}. \quad (5)$$

Trong báo cáo này, nếu không nêu rõ, $\text{sup}(X)$ được hiểu là độ hỗ trợ tuyệt đối. Với ngưỡng hỗ trợ tối thiểu minsup được biểu diễn theo số giao dịch, X là một tập mục phổ biến (*frequent itemset*) khi:

$$\text{sup}(X) \geq \text{minsup}. \quad (6)$$

Do đó, bài toán FIM được phát biểu:

$$FI(D, \text{minsup}) = \{X \subseteq I \mid \text{sup}(X) \geq \text{minsup}\}. \quad (7)$$

Bên cạnh FI thông thường, hai dạng biểu diễn rút gọn thường được xét là tập mục phổ biến đóng (*closed frequent itemset*) và tập mục phổ biến cực đại (*maximal frequent itemset*) [2].

Một FI X là CFI nếu không tồn tại FI Y thỏa:

$$X \subset Y \quad \text{và} \quad \text{sup}(X) = \text{sup}(Y). \quad (8)$$

Một FI X là MFI nếu không tồn tại FI Y thỏa:

$$X \subset Y. \quad (9)$$

Quan hệ giữa ba tập kết quả là:

$$MFI \subseteq CFI \subseteq FI. \quad (10)$$

MFI cho kết quả gọn hơn nhưng không bảo toàn trực tiếp support của mọi tập con. CFI vẫn là biểu diễn không dư thừa theo quan hệ support và có thể được dùng để suy ra support của các FI thông thường.

1.2 Tính chất Apriori

Tính chất Apriori, còn gọi là tính chất phản đơn điệu (*anti-monotonicity*) của support, phát biểu rằng với hai itemset $X \subseteq Y$:

$$\text{sup}(Y) \leq \text{sup}(X). \quad (11)$$

Thật vậy, mọi giao dịch chứa Y đều chứa X , nên:

$$\{T_j \in D \mid Y \subseteq T_j\} \subseteq \{T_j \in D \mid X \subseteq T_j\}. \quad (12)$$

Từ đó thu được hai hệ quả quan trọng:

- Nếu X không phổ biến thì mọi tập cha $Y \supset X$ cũng không phổ biến.
- Nếu Y phổ biến thì mọi tập con $X \subseteq Y$ đều phổ biến.

Apriori sử dụng trực tiếp tính chất này để sinh và loại ứng viên theo từng mức kích thước [1]. Các thuật toán về sau như Eclat, FP-Growth và PrePost vẫn dựa trên tính phản đơn điệu để tránh không gian tìm kiếm, nhưng sử dụng các cấu trúc biểu diễn và chiến lược khai thác khác nhau [3], [4], [5].

1.3 Tổng quan thuật toán PrePost

PrePost là thuật toán khai thác toàn bộ FI dựa trên cấu trúc *Pre-order Post-order Coding tree* và danh sách mã hóa node [5]. Thuật toán được thiết kế để tránh việc quét lại TDB trong giai đoạn mở rộng itemset và giảm chi phí biểu diễn support so với việc lưu trực tiếp danh sách TID dài.

Quy trình tổng quát gồm:

1. Quét TDB để tính support của từng item và loại các item không phổ biến.
2. Sắp xếp các item phổ biến theo một thứ tự toàn cục, thường là support giảm dần.
3. Chèn các giao dịch đã lọc và sắp xếp vào PPC-tree.
4. Duyệt cây để gán mã *pre-order* và *post-order*.
5. Xây dựng N-list cho các frequent 1-itemset.
6. Mở rộng itemset theo chiều sâu bằng phép giao N-list và loại các itemset có support nhỏ hơn *minsup*.

Với hai node u và v trên PPC-tree, u là tổ tiên của v khi:

$$pre(u) < pre(v) \quad \text{và} \quad post(u) > post(v). \quad (13)$$

Điều kiện này cho phép kiểm tra quan hệ tổ tiên-hậu duệ chỉ bằng hai số nguyên, không cần duyệt lại đường đi trên cây. Đây là cơ sở để PrePost tính support của các itemset mở rộng thông qua N-list [5].

1.4 Cấu trúc PPC-tree

PPC-tree là cây tiền tổ được xây dựng từ TDB sau khi loại các item không phổ biến và sắp xếp các item còn lại theo thứ tự toàn cục. Việc đặt các item có support cao ở gần gốc thường làm tăng khả năng chia sẻ tiền tổ giữa các giao dịch, từ đó giảm số node cần lưu.

Mỗi node PPC-tree lưu:

- **item-name:** item được biểu diễn bởi node;
- **count:** số giao dịch đi qua tiền tổ kết thúc tại node;
- **children:** tập hoặc danh sách node con;
- **pre:** thứ tự node khi được thăm trong phép duyệt trước;
- **post:** thứ tự node khi hoàn tất trong phép duyệt sau.

Mỗi node được gán một cặp mã:

$$PPCode(u) = (pre(u), post(u)). \quad (14)$$

Các giá trị pre và $post$ phải được sinh nhất quán bởi cùng một quá trình duyệt cây. Giá trị cụ thể không quan trọng bằng việc chúng bảo toàn điều kiện tổ tiên trong Công thức (13).

1.5 Cấu trúc N-list

N-list của một item hoặc itemset là một danh sách các bộ ba:

$$(pre, post, count). \quad (15)$$

Với frequent 1-itemset $\{i\}$, $NList(i)$ chứa PP-code và count của tất cả node mang nhãn i . Support được tính bằng:

$$sup(i) = \sum_{e \in NList(i)} e.count. \quad (16)$$

Với itemset dài hơn, N-list được xây dựng từ N-list của hai tiền tố có quan hệ phù hợp trong cây liệt kê tập hợp. Phép kết hợp chỉ giữ các cặp node thỏa quan hệ tổ tiên-hậu duệ theo mã pre/post. Khi nhiều phần tử tạo ra cùng một PP-code đầu ra, count của chúng phải được cộng gộp. Có thể ký hiệu phép này dưới dạng:

$$NList(P \cup \{x\}) = NList(P) \bowtie_N NList(x), \quad (17)$$

trong đó $\bowtie_N$ là phép giao N-list theo định nghĩa và thứ tự item của PrePost [5].

Sau khi giao, support của itemset được tính trực tiếp từ N-list:

$$sup(P \cup \{x\}) = \sum_{e \in NList(P \cup \{x\})} e.count. \quad (18)$$

Điểm cần lưu ý là N-list của một itemset không nhất thiết chỉ có một phần tử. Số phần tử phụ thuộc vào số node PPC-tree thỏa quan hệ cần thiết; các phần tử chỉ được hợp nhất khi chúng có cùng PP-code đầu ra.

Nhờ N-list, sau khi PPC-tree được xây dựng, thuật toán không cần quét lại TDB để tính support cho từng itemset mở rộng. Phần lớn chi phí được chuyển thành các phép duyệt và giao trên các danh sách mã hóa node [5].

1.6 Giải mã thuật toán PrePost

Thuật toán 1 mô tả quy trình PrePost ở mức tổng quát. Giải mã tách rõ hai giai đoạn: tiền xử lý để xây PPC-tree và N-list, sau đó khai thác theo chiều sâu.

Listing 1: Giải mã tổng quát của thuật toán PrePost

```

1 Input : Transaction database D, minimum support minsup
2 Output: All frequent itemsets FI
3
4 1. Scan D and compute the support of every item.
5 2. Remove items having support < minsup.
6 3. Build a global order F of frequent items by descending support
   .
7 4. Create an empty PPC-tree.
8 5. For each transaction T in D:
9     a. Remove infrequent items from T.
10    b. Sort the remaining items according to F.
11    c. Insert the ordered transaction into the PPC-tree.
12 6. Assign pre-order and post-order codes to all PPC-tree nodes.
13 7. Build the N-list of every frequent 1-itemset.
14 8. Call Mine(prefix = empty, extensions = frequent 1-itemsets).
15
16 Procedure Mine(prefix, extensions):
17 1. For each extension X in extensions:
18     a. P = prefix union {X}.
19     b. Output P.
20     c. next_extensions = empty.
21     d. For each extension Y occurring after X:
22         i. Construct NList(P union {Y}) by N-list
           intersection.
23         ii. Compute support from the resulting N-list.
24         iii. If support >= minsup:
25             store the N-list;
26             add Y to next_extensions.
27     e. If next_extensions is not empty:
28         Mine(P, next_extensions).

```

Thủ tục Mine duyệt không gian tìm kiếm theo chiều sâu. Một itemset mở rộng chỉ được đưa vào mức đệ quy tiếp theo khi support của nó không nhỏ hơn *minsup*. Việc loại sớm này là hệ quả trực tiếp của tính phản đơn điệu: nếu itemset hiện tại không phổ biến thì mọi phần mở rộng của nó đều không phổ biến.

1.7 Phân tích độ phức tạp

Hiệu năng của PrePost phụ thuộc vào số giao dịch, số item phân biệt, độ dài giao dịch, ngưỡng *minsup*, mức độ chia sẻ tiền tố và số FI trong kết quả.

Gọi $n = |D|$, $m = |I|$ và ℓ là độ dài trung bình của giao dịch sau lọc. Quét TDB để đếm support của item đơn có chi phí:

$$O(n\ell). \quad (19)$$

Nếu các item trong mỗi giao dịch cần được sắp xếp độc lập, chi phí tiền xử lý có thể bao gồm:

$$O\left(\sum_{T \in D} |T| \log |T|\right). \quad (20)$$

Sau khi giao dịch đã được sắp xếp, việc chèn vào PPC-tree xử lý mỗi lần xuất hiện item ít nhất một lần. Khi phép tra cứu node con được cài đặt bằng bảng băm hoặc ánh xạ phù hợp, chi phí xây cây thường gần:

$$O(n\ell). \quad (21)$$

Tuy nhiên, không thể biểu diễn toàn bộ độ phức tạp khai thác chỉ bằng n , m và ℓ , vì chi phí còn phụ thuộc vào số itemset được xét và độ dài các N-list. Nếu một phép giao hai N-list được cài đặt bằng hai con trỏ trên các danh sách đã sắp xếp, chi phí của phép giao thường tỉ lệ với tổng độ dài hai danh sách:

$$O(|NList(X)| + |NList(Y)|). \quad (22)$$

Tổng thời gian khai thác có thể mô tả ở mức tổng quát:

$$O\left(\sum_{c \in \mathcal{C}} \text{costJoin}(c)\right), \quad (23)$$

trong đó $\mathcal{C}$ là tập các phép mở rộng thực sự được thuật toán xét.

Trong trường hợp xấu nhất, số FI có thể tăng theo hàm mũ theo số item. Đây là giới hạn đầu ra của bài toán FIM chứ không chỉ riêng PrePost. Vì vậy, khi dữ liệu dày đặc và *minsup* thấp, thời gian chạy và bộ nhớ có thể tăng mạnh do số itemset và N-list trung gian rất lớn [2], [6].

Độ phức tạp không gian gồm:

- bộ nhớ lưu PPC-tree;
- bộ nhớ lưu N-list của frequent 1-itemset;
- bộ nhớ cho các N-list trung gian trên các nhánh khai thác đang hoạt động;

- bộ nhớ lưu kết quả, nếu toàn bộ FI được giữ trong RAM.

Các yếu tố ảnh hưởng chính đến hiệu năng là:

1. **Ngưỡng *minsup***: ngưỡng thấp thường tạo nhiều FI và nhiều phép giao N-list hơn.
2. **Độ dày dữ liệu**: dữ liệu dày đặc có xu hướng tạo nhiều itemset dài.
3. **Độ dài giao dịch**: giao dịch dài làm tăng chi phí xây cây và số khả năng kết hợp.
4. **Mức độ chia sẻ tiền tố**: chia sẻ cao giúp PPC-tree nén tốt hơn.
5. **Cài đặt N-list**: cách tổ chức dữ liệu, giao danh sách, cộng gộp count và quản lý cập phát ảnh hưởng trực tiếp đến thời gian chạy.
6. **Chiến lược lưu ứng viên**: loại ngay itemset không đạt *minsup* giúp tránh lưu và mở rộng các nhánh chắc chắn thất bại.

1.8 So sánh với các thuật toán FIM tiêu biểu

Apriori. Apriori khai thác theo chiều rộng và sinh candidate k -itemset từ các frequent $(k - 1)$ -itemset. Thuật toán dễ hiểu và dễ kiểm chứng, nhưng có thể tạo rất nhiều ứng viên và cần quét TDB qua nhiều mức [1], [2].

Eclat. Eclat sử dụng biểu diễn dọc, trong đó mỗi item hoặc itemset được gắn với một tidset. Support của itemset mở rộng được tính bằng phép giao tidset. Phương pháp này phù hợp với khai thác theo chiều sâu và tránh quét lại TDB, nhưng các tidset có thể lớn khi số giao dịch cao [3].

FP-Growth. FP-Growth xây dựng FP-tree để nén TDB và khai thác mẫu bằng chiến lược tăng trưởng mẫu, tránh sinh candidate theo từng mức như Apriori [4]. Trong quá trình khai thác, thuật toán sử dụng cơ sở mẫu điều kiện và các FP-tree điều kiện; chi phí của các cấu trúc điều kiện phụ thuộc vào đặc trưng dữ liệu.

PrePost. PrePost cũng sử dụng cây tiền tố, nhưng sau khi xây PPC-tree, thuật toán mã hóa các node bằng pre/post và biểu diễn itemset bằng N-list. Việc mở rộng itemset được thực hiện qua phép giao N-list thay vì xây dựng lặp lại các cây điều kiện [5]. So với Apriori, PrePost tránh sinh ứng viên theo từng mức và tránh quét TDB nhiều lần. So với Eclat, PrePost thay tidset bằng biểu diễn node nén theo PPC-tree. So với FP-Growth, PrePost chuyển phần lớn giai đoạn khai thác sang thao tác danh sách.

PrePost không luôn vượt trội trên mọi loại dữ liệu. Khi dữ liệu ít chia sẻ tiền tố hoặc *minsup* rất thấp, PPC-tree và các N-list trung gian vẫn có thể lớn. Hiệu năng thực tế còn phụ thuộc mạnh vào cấu trúc dữ liệu và cách cài đặt phép giao N-list.

PrePost+ tiếp tục cải tiến PrePost bằng chiến lược *Children-Parent Equivalence pruning* để giảm số nhánh cần khai thác [7]. Trong phần thực nghiệm của báo cáo, SPMF được dùng như một triển khai tham chiếu vì đây là thư viện Java mã nguồn mở chuyên về khai phá mẫu và cung cấp nhiều thuật toán FIM [8], [9]. Các bộ dữ liệu chuẩn được lấy từ FIMI Repository nhằm hỗ trợ so sánh trên những tập dữ liệu thường dùng trong nghiên cứu FIM [10], [11].

1.9 Kết luận chương

Chương này đã trình bày bài toán FIM, các khái niệm support, FI, CFI, MFI và tính chất Apriori. Trên cơ sở đó, chương giới thiệu thuật toán PrePost với hai cấu trúc cốt lõi là PPC-tree và N-list, mô tả quy trình khai thác, giả mã và các yếu tố chi phối độ phức tạp. Phần so sánh cho thấy PrePost thuộc nhóm thuật toán khai thác theo chiều sâu, tránh quét lại TDB bằng cách mã hóa thông tin support trên cây và thực hiện các phép giao N-list. Đây là nền tảng để chương tiếp theo minh họa quá trình xây PPC-tree, gán mã pre/post và sinh N-list trên một ví dụ cụ thể.

2 Ví dụ minh họa thuật toán PrePost

Phần này trình bày hai ví dụ tính toán thủ công nhằm minh họa các cấu trúc và phép toán cốt lõi của PrePost [5]. Ví dụ thứ nhất mô tả đầy đủ quy trình từ cơ sở dữ liệu giao dịch ban đầu, tính support, xây dựng PPC-tree, gán mã *pre/post*, tạo N-list và khai thác các frequent itemset. Ví dụ thứ hai xét trường hợp PPC-tree chỉ gồm một nhánh, qua đó minh họa khả năng nén tiền tố và tính chất đường đơn của N-list.

Trong toàn bộ chương, $\text{sup}(X)$ là support tuyệt đối và minsup cũng được biểu diễn theo số giao dịch. Thứ tự item và quy ước giao N-list được cố định như mô tả trong từng ví dụ để các bước tính toán nhất quán.

2.1 Ví dụ 1: Ví dụ cơ sở

2.1.1 Cơ sở dữ liệu giao dịch ban đầu

Xét cơ sở dữ liệu giao dịch gồm 5 giao dịch và 5 item phân biệt. Ngưỡng hỗ trợ tối thiểu được chọn là:

$$\text{minsup} = 2. \quad (24)$$

Cơ sở dữ liệu giao dịch ban đầu được trình bày trong Bảng 2.

Bảng 2: Cơ sở dữ liệu giao dịch của ví dụ cơ sở

TID	Giao dịch
T_1	{1, 2, 3}
T_2	{1, 3, 4}
T_3	{2, 3, 5}
T_4	{1, 2, 3, 5}
T_5	{2, 3, 4}

2.1.2 Bước 1: Tính support của các item đơn

Thuật toán quét cơ sở dữ liệu một lần để tính support của từng item. Kết quả được trình bày trong Bảng 3.

Bảng 3: Support của các item đơn trong ví dụ cơ sở

Item	Support
1	3
2	4
3	5
4	2
5	2

Vì tất cả các item đều có support không nhỏ hơn $minsup$, toàn bộ 5 item đều được giữ lại.

Theo quy trình xây dựng PPC-tree, các item phổ biến được sắp xếp theo một thứ tự toàn cục [5]. Trong ví dụ này, nhóm chọn support giảm dần; nếu hai item có cùng support thì mã item tăng dần được dùng để phá hòa. Do đó, thứ tự toàn cục là:

$$3 \succ 2 \succ 1 \succ 4 \succ 5. \quad (25)$$

2.1.3 Bước 2: Sắp xếp lại các giao dịch

Các item trong mỗi giao dịch được sắp xếp lại theo thứ tự toàn cục trong Công thức 25. Kết quả được trình bày trong Bảng 4.

Bảng 4: Cơ sở dữ liệu giao dịch sau khi sắp xếp

TID	Giao dịch ban đầu	Giao dịch sau sắp xếp
T_1	$\{1, 2, 3\}$	$\langle 3, 2, 1 \rangle$
T_2	$\{1, 3, 4\}$	$\langle 3, 1, 4 \rangle$
T_3	$\{2, 3, 5\}$	$\langle 3, 2, 5 \rangle$
T_4	$\{1, 2, 3, 5\}$	$\langle 3, 2, 1, 5 \rangle$
T_5	$\{2, 3, 4\}$	$\langle 3, 2, 4 \rangle$

2.1.4 Bước 3: Xây dựng PPC-tree

Các giao dịch đã sắp xếp được chèn lần lượt vào PPC-tree.

Chèn $T_1 = \langle 3, 2, 1 \rangle$: tạo đường đi $3 \rightarrow 2 \rightarrow 1$, trong đó mỗi node có *count* = 1.

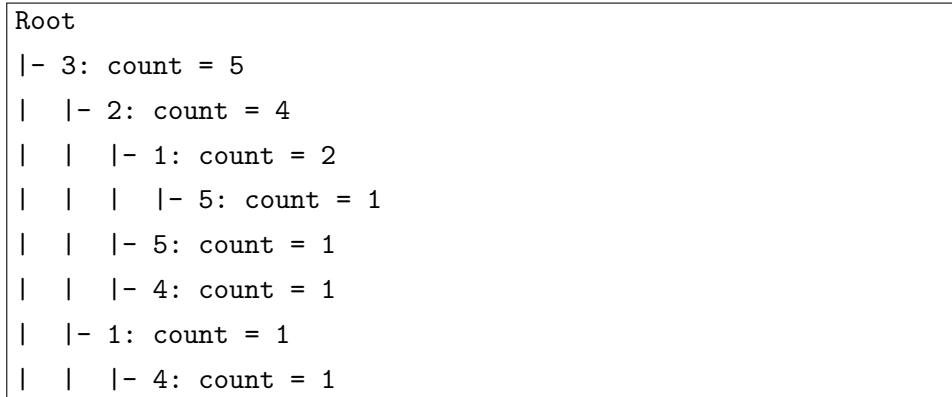
Chèn $T_2 = \langle 3, 1, 4 \rangle$: node 3 đã tồn tại nên tăng *count*(3) lên 2. Sau đó tạo nhánh mới $1 \rightarrow 4$ dưới node 3.

Chèn $T_3 = \langle 3, 2, 5 \rangle$: tăng *count*(3) lên 3, tăng *count*(2) dưới node 3 lên 2, sau đó tạo node 5 dưới node 2.

Chèn $T_4 = \langle 3, 2, 1, 5 \rangle$: tăng *count*(3) lên 4, tăng *count*(2) lên 3, tăng *count*(1) dưới node 2 lên 2, sau đó tạo node 5 dưới node 1.

Chèn $T_5 = \langle 3, 2, 4 \rangle$: tăng *count*(3) lên 5, tăng *count*(2) lên 4, sau đó tạo node 4 dưới node 2.

PPC-tree sau khi chèn toàn bộ giao dịch được minh họa trong Hình 1.



Hình 1: PPC-tree của ví dụ cơ sở trước khi gán mã *pre* và *post*

2.1.5 Bước 4: Gán mã *pre* và *post*

Mã *pre* được gán khi node được thăm lần đầu trong phép duyệt tiên thứ tự, còn mã *post* được gán khi toàn bộ cây con của node đã được xử lý. Cặp mã này bảo toàn quan hệ tổ tiên-hậu duệ trên PPC-tree [5].

Kết quả được trình bày trong Bảng 5.

Bảng 5: Các node của PPC-tree sau khi gán mã *pre* và *post*

Ký hiệu node	Item	Count	<i>pre</i>	<i>post</i>
n_1	3	5	1	8
n_2	2	4	2	5
n_3	1	2	3	2
n_4	5	1	4	1
n_5	5	1	5	3
n_6	4	1	6	4
n_7	1	1	7	7
n_8	4	1	8	6

PPC-tree sau khi gán mã *pre* và *post* được minh họa trong Hình 2.

Root
- 3: count = 5, pre = 1, post = 8
- 2: count = 4, pre = 2, post = 5
- 1: count = 2, pre = 3, post = 2
- 5: count = 1, pre = 4, post = 1
- 5: count = 1, pre = 5, post = 3
- 4: count = 1, pre = 6, post = 4
- 1: count = 1, pre = 7, post = 7
- 4: count = 1, pre = 8, post = 6

Hình 2: PPC-tree của ví dụ cơ sở sau khi gán mã *pre* và *post*

2.1.6 Bước 5: Tạo N-list cho các item đơn

Mỗi phần tử trong N-list có dạng:

$$(pre, post, count). \quad (26)$$

N-list của một item được xây dựng bằng cách thu thập các PP-code và count của tất cả node mang item đó trong PPC-tree [5].

Kết quả được trình bày trong Bảng 6.

Bảng 6: N-list của các item đơn trong ví dụ cơ sở

Item	N-list	Support
3	$\{(1, 8, 5)\}$	5
2	$\{(2, 5, 4)\}$	4
1	$\{(3, 2, 2), (7, 7, 1)\}$	3
4	$\{(6, 4, 1), (8, 6, 1)\}$	2
5	$\{(4, 1, 1), (5, 3, 1)\}$	2

Support của một item được tính bằng tổng các giá trị *count* trong N-list. Ví dụ:

$$sup(1) = 2 + 1 = 3. \quad (27)$$

2.1.7 Bước 6: Sinh frequent 2-itemset bằng N-list

Hai node x và y có quan hệ tổ tiên–hậu duệ nếu:

$$pre(x) < pre(y) \quad \text{và} \quad post(x) > post(y). \quad (28)$$

Trong quy ước được sử dụng cho ví dụ này, khi tạo N-list của cặp item (a, b) theo thứ tự toàn cục, thuật toán tìm node của a là tổ tiên của node b . Phần tử kết quả giữ PP-code của node tổ tiên và nhận count từ node hậu duệ. Nếu nhiều node hậu duệ được ánh xạ vào cùng một PP-code tổ tiên, các count được cộng gộp. Đây là phép giao N-list, qua đó support của itemset được tính mà không cần quét lại TDB [5].

Cần lưu ý rằng cách viết thứ tự các item trong ký hiệu $NList(a, b)$ phải được giữ nhất quán với quy ước triển khai. Bản chất của phép toán là kiểm tra quan hệ tổ tiên–hậu duệ và cộng gộp đúng trường count, không phải chỉ ghép hai danh sách theo vị trí.

Ví dụ, để tạo N-list của itemset $\{2, 3\}$, xét:

$$NList(3) = \{(1, 8, 5)\}, \quad (29)$$

và:

$$NList(2) = \{(2, 5, 4)\}. \quad (30)$$

Vì:

$$1 < 2 \quad \text{và} \quad 8 > 5, \quad (31)$$

node của item 3 là tổ tiên của node item 2. Do đó:

$$NList(3, 2) = \{(1, 8, 4)\}. \quad (32)$$

Support của itemset được tính bằng tổng trường *count* trong N-list:

$$sup(\{2, 3\}) = 4. \quad (33)$$

Các frequent 2-itemset thu được được trình bày trong Bảng 7.

Bảng 7: Các frequent 2-itemset trong ví dụ cơ sở

Itemset	N-list theo thứ tự toàn cục	Support
$\{1, 2\}$	$NList(2, 1) = \{(2, 5, 2)\}$	2
$\{1, 3\}$	$NList(3, 1) = \{(1, 8, 3)\}$	3
$\{2, 3\}$	$NList(3, 2) = \{(1, 8, 4)\}$	4
$\{2, 5\}$	$NList(2, 5) = \{(2, 5, 2)\}$	2
$\{3, 4\}$	$NList(3, 4) = \{(1, 8, 2)\}$	2
$\{3, 5\}$	$NList(3, 5) = \{(1, 8, 2)\}$	2

Chẳng hạn, hai node của item 5 là $(4, 1, 1)$ và $(5, 3, 1)$ đều là hậu duệ của node item 2 có mã $(2, 5, 4)$. Do đó:

$$NList(2, 5) = \{(2, 5, 1 + 1)\} = \{(2, 5, 2)\}. \quad (34)$$

Các 2-itemset còn lại có support nhỏ hơn *minsup* nên bị loại.

2.1.8 Bước 7: Sinh frequent 3-itemset

Đối với itemset dài hơn hai, N-list được tạo từ N-list của các itemset con có chung phần hậu tố theo cây liệt kê tập hợp của PrePost [5]. Với quy ước ký hiệu trong ví dụ, N-list của (i_x, i_y, S) được tạo từ N-list của (i_x, S) và (i_y, S) . Sau mỗi phép giao, support được tính bằng tổng count; chỉ itemset đạt *minsup* mới được giữ lại để tiếp tục mở rộng.

Từ các frequent 2-itemset, hai frequent 3-itemset thu được là:

$$\{1, 2, 3\} \quad \text{và} \quad \{2, 3, 5\}.$$

Itemset $\{1, 2, 3\}$. Theo thứ tự toàn cục, itemset được biểu diễn là $(3, 2, 1)$. Ta sử dụng:

$$NList(3, 1) = \{(1, 8, 3)\}, \quad (35)$$

và:

$$NList(2, 1) = \{(2, 5, 2)\}. \quad (36)$$

Vì node có mã (1, 8) là tổ tiên của node có mã (2, 5), phần tử kết quả giữ mã của node tổ tiên và lấy count bằng 2 từ node hậu duệ:

$$NList(3, 2, 1) = \{(1, 8, 2)\}. \quad (37)$$

Do đó:

$$sup(\{1, 2, 3\}) = 2. \quad (38)$$

Itemset {2, 3, 5}. Theo thứ tự toàn cục, itemset được biểu diễn là (3, 2, 5). Ta sử dụng:

$$NList(3, 5) = \{(1, 8, 2)\}, \quad (39)$$

và:

$$NList(2, 5) = \{(2, 5, 2)\}. \quad (40)$$

Vì node có mã (1, 8) là tổ tiên của node có mã (2, 5), ta thu được:

$$NList(3, 2, 5) = \{(1, 8, 2)\}. \quad (41)$$

Do đó:

$$sup(\{2, 3, 5\}) = 2. \quad (42)$$

Các frequent 3-itemset được trình bày trong Bảng 8.

Bảng 8: Các frequent 3-itemset trong ví dụ cơ sở

Itemset	N-list theo thứ tự toàn cục	Support
{1, 2, 3}	$NList(3, 2, 1) = \{(1, 8, 2)\}$	2
{2, 3, 5}	$NList(3, 2, 5) = \{(1, 8, 2)\}$	2

Ứng viên 4-itemset có thể được tạo từ hai frequent 3-itemset trên là:

$$\{1, 2, 3, 5\}.$$

Itemset này chỉ xuất hiện trong giao dịch T_4 , do đó:

$$sup(\{1, 2, 3, 5\}) = 1 < minsup. \quad (43)$$

Vì vậy, không tồn tại frequent itemset có kích thước từ 4 trở lên.

2.1.9 Tập kết quả cuối cùng

Tập frequent itemset cuối cùng của ví dụ cơ sở được trình bày trong Bảng 9.

Bảng 9: Tập frequent itemset cuối cùng của ví dụ cơ sở

Itemset	Support
$\{1\}$	3
$\{2\}$	4
$\{3\}$	5
$\{4\}$	2
$\{5\}$	2
$\{1, 2\}$	2
$\{1, 3\}$	3
$\{2, 3\}$	4
$\{2, 5\}$	2
$\{3, 4\}$	2
$\{3, 5\}$	2
$\{1, 2, 3\}$	2
$\{2, 3, 5\}$	2

2.1.10 Kiểm tra chéo bằng liệt kê thủ công

Để kiểm tra kết quả, support của các itemset được tính lại trực tiếp từ cơ sở dữ liệu giao dịch.

Support của toàn bộ 2-itemset được trình bày trong Bảng 10.

Bảng 10: Kiểm tra chéo support của toàn bộ 2-itemset

Itemset	Các giao dịch chứa itemset	Support
$\{1, 2\}$	T_1, T_4	2
$\{1, 3\}$	T_1, T_2, T_4	3
$\{1, 4\}$	T_2	1
$\{1, 5\}$	T_4	1
$\{2, 3\}$	T_1, T_3, T_4, T_5	4
$\{2, 4\}$	T_5	1
$\{2, 5\}$	T_3, T_4	2
$\{3, 4\}$	T_2, T_5	2
$\{3, 5\}$	T_3, T_4	2
$\{4, 5\}$	Không có	0

Support của toàn bộ 3-itemset được trình bày trong Bảng 11.

Bảng 11: Kiểm tra chéo support của toàn bộ 3-itemset

Itemset	Các giao dịch chứa itemset	Support
$\{1, 2, 3\}$	T_1, T_4	2
$\{1, 2, 4\}$	Không có	0
$\{1, 2, 5\}$	T_4	1
$\{1, 3, 4\}$	T_2	1
$\{1, 3, 5\}$	T_4	1
$\{1, 4, 5\}$	Không có	0
$\{2, 3, 4\}$	T_5	1
$\{2, 3, 5\}$	T_3, T_4	2
$\{2, 4, 5\}$	Không có	0
$\{3, 4, 5\}$	Không có	0

Không có itemset nào có kích thước từ 4 trở lên đạt $minsup = 2$. Vì vậy, kết quả kiểm tra chéo khớp với tập frequent itemset được khai thác bằng PrePost.

2.2 Ví dụ 2: Trường hợp PPC-tree một nhánh

2.2.1 Cơ sở dữ liệu giao dịch ban đầu

Ví dụ thứ hai được thiết kế sao cho PPC-tree chỉ gồm một nhánh. Đây là trường hợp các giao dịch có mức độ chia sẻ tiền tố cao, giúp PPC-tree nén dữ liệu hiệu quả.

Ngưỡng hỗ trợ tối thiểu vẫn được chọn là:

$$minsup = 2. \quad (44)$$

Cơ sở dữ liệu giao dịch được trình bày trong Bảng 12.

Bảng 12: Cơ sở dữ liệu giao dịch của ví dụ một nhánh

TID	Giao dịch
T_1	$\{1, 2, 3, 4\}$
T_2	$\{1, 2, 3\}$
T_3	$\{1, 2, 3, 4\}$
T_4	$\{1, 2, 3\}$
T_5	$\{1, 2\}$

2.2.2 Bước 1: Tính support và xác định thứ tự toàn cục

Support của các item đơn được trình bày trong Bảng 13.

Bảng 13: Support của các item đơn trong ví dụ một nhánh

Item	Support
1	5
2	5
3	4
4	2

Tất cả các item đều phổ biến. Thứ tự toàn cục được chọn là:

$$1 \succ 2 \succ 3 \succ 4. \quad (45)$$

Sau khi sắp xếp, các giao dịch được biểu diễn như trong Bảng 14.

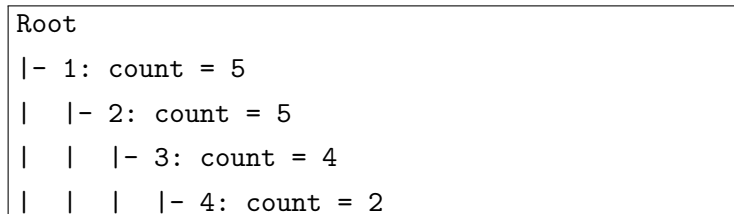
Bảng 14: Cơ sở dữ liệu giao dịch sau khi sắp xếp

TID	Giao dịch sau sắp xếp
T_1	$\langle 1, 2, 3, 4 \rangle$
T_2	$\langle 1, 2, 3 \rangle$
T_3	$\langle 1, 2, 3, 4 \rangle$
T_4	$\langle 1, 2, 3 \rangle$
T_5	$\langle 1, 2 \rangle$

2.2.3 Bước 2: Xây dựng PPC-tree

Tất cả các giao dịch đều chia sẻ tiền tố $1 \rightarrow 2$. Bốn giao dịch chứa item 3 tiếp tục đi qua node 3, trong khi hai giao dịch chứa item 4 tiếp tục đi qua node 4.

Do đó, PPC-tree chỉ gồm một nhánh như Hình 3.



Hình 3: PPC-tree một nhánh trong ví dụ đặc biệt

2.2.4 Bước 3: Gán mã pre và post

Vì cây chỉ có một nhánh, mã *pre* tăng dần từ node gần gốc xuống node lá, trong khi mã *post* tăng dần từ node lá lên node gần gốc.

Kết quả được trình bày trong Bảng 15.

Bảng 15: Mã *pre* và *post* trong PPC-tree một nhánh

Ký hiệu node	Item	Count	<i>pre</i>	<i>post</i>
n_1	1	5	1	4
n_2	2	5	2	3
n_3	3	4	3	2
n_4	4	2	4	1

2.2.5 Bước 4: Tạo N-list

Vì mỗi item chỉ xuất hiện tại một node, N-list của mỗi item chỉ gồm một phần tử.

Bảng 16: N-list của các item đơn trong ví dụ một nhánh

Item	N-list	Support
1	$\{(1, 4, 5)\}$	5
2	$\{(2, 3, 5)\}$	5
3	$\{(3, 2, 4)\}$	4
4	$\{(4, 1, 2)\}$	2

2.2.6 Bước 5: Khai thác các frequent itemset

Trong PPC-tree một nhánh, mọi node đứng trước đều là tổ tiên của các node đứng sau. PrePost khai thác trường hợp này bằng tính chất đường đơn của N-list, cho phép suy ra trực tiếp nhiều frequent itemset mà không cần thực hiện đầy đủ mọi phép sinh và kiểm tra ứng viên [5].

Riêng trong bộ dữ liệu của ví dụ này, các giao dịch tạo thành các tiền tổ lồng nhau. Vì vậy, support của một itemset bằng count của node sâu nhất tương ứng với item xuất hiện muộn nhất trong thứ tự toàn cục.

Ví dụ, với itemset $\{1, 2, 3, 4\}$, node sâu nhất là node item 4 có *count* = 2. Do đó:

$$sup(\{1, 2, 3, 4\}) = 2. \quad (46)$$

Vì support bằng *minsup*, itemset này là frequent itemset.

Toàn bộ frequent itemset được trình bày trong Bảng 17.

Bảng 17: Tập frequent itemset cuối cùng của ví dụ một nhánh

Itemset	Support
{1}	5
{2}	5
{3}	4
{4}	2
{1, 2}	5
{1, 3}	4
{1, 4}	2
{2, 3}	4
{2, 4}	2
{3, 4}	2
{1, 2, 3}	4
{1, 2, 4}	2
{1, 3, 4}	2
{2, 3, 4}	2
{1, 2, 3, 4}	2

2.2.7 Phân tích trường hợp PPC-tree một nhánh

Trường hợp PPC-tree một nhánh cho thấy khả năng nén của PPC-tree khi nhiều giao dịch chia sẻ cùng một tiền tố, đồng thời minh họa tính chất đường đơn được PrePost sử dụng trong quá trình khai thác [5].

Trong ví dụ này, thay vì lưu riêng 5 giao dịch, PPC-tree chỉ cần 4 node để biểu diễn toàn bộ cơ sở dữ liệu. Giá trị count tại mỗi node cho biết số giao dịch đi qua tiền tố tương ứng.

Việc khai thác frequent itemset cũng trở nên đơn giản hơn vì quan hệ tổ tiên – hậu duệ giữa các node luôn được xác định trực tiếp theo vị trí trên nhánh. Đối với mọi itemset trên nhánh, support được xác định bởi count của node sâu nhất trong itemset.

Tuy nhiên, ví dụ này cũng cho thấy hiện tượng bùng nổ số lượng frequent itemset trong dữ liệu dày đặc. Với 4 item và $minsup = 2$, toàn bộ các tập con không rỗng đều phổ biến, tạo ra:

$$2^4 - 1 = 15 \quad (47)$$

frequent itemset.

Do đó, khả năng nén tốt của cây không loại bỏ giới hạn do kích thước đầu ra: thời gian khai thác vẫn có thể tăng mạnh khi số frequent itemset là rất lớn. Đây là đặc điểm chung của bài toán FIM trên dữ liệu dày đặc hoặc khi ngưỡng support thấp [2].

2.3 Kết luận

Hai ví dụ trên minh họa các bước chính của thuật toán PrePost [5].

Ví dụ cơ sở trình bày đầy đủ quá trình tính support, sắp xếp giao dịch, xây dựng PPC-tree, gán mã *pre/post*, tạo N-list và khai thác frequent itemset. Kết quả được kiểm tra chéo bằng cách tính support trực tiếp từ cơ sở dữ liệu giao dịch.

Ví dụ thứ hai minh họa trường hợp PPC-tree một nhánh, trong đó các giao dịch chia sẻ tiền tố cao. Trường hợp này cho thấy PPC-tree có khả năng nén dữ liệu hiệu quả, nhưng đồng thời cũng cho thấy số lượng frequent itemset có thể tăng nhanh khi dữ liệu dày đặc hoặc ngưỡng *minsup* thấp.

3 Cài đặt thuật toán PrePost

Chương này trình bày quá trình cài đặt thuật toán PrePost bằng ngôn ngữ Julia [12]. Mục tiêu là hiện thực các thành phần cốt lõi của thuật toán gốc [5]: đọc cơ sở dữ liệu giao dịch, tiền xử lý theo ngưỡng *minsup*, xây dựng PPC-tree, gán mã *pre/post*, tạo N-list, khai thác toàn bộ frequent itemset và xuất kết quả theo định dạng có thể đối chiếu với thư viện SPMF [8], [9].

Việc cài đặt được tổ chức theo bốn mức yêu cầu. Level 1 tập trung vào tính đúng đắn của thuật toán cơ bản. Level 2 bổ sung kiểm thử tự động trên nhiều CSDL nhỏ. Level 3 cài đặt một phiên bản tối ưu và đối chiếu với phiên bản cơ bản. Level 4 hoàn thiện phần đọc/ghi file và giao diện dòng lệnh.

3.1 Môi trường và công cụ

Thuật toán được cài đặt bằng Julia, một ngôn ngữ hướng tới khả năng biểu đạt ở mức cao đồng thời hỗ trợ biên dịch chuyên biệt theo kiểu dữ liệu [12]. Project sử dụng Julia từ phiên bản 1.9 trở lên và có module chính là **PrePostFIM**. Việc chạy chương trình, kiểm thử và đối chiếu kết quả được thực hiện thông qua các script dòng lệnh trên Windows PowerShell.

Môi trường cài đặt được tóm tắt trong Bảng 24.

Bảng 18: Môi trường và công cụ cài đặt

Thành phần	Mô tả
Ngôn ngữ lập trình	Julia
Module chính	PrePostFIM
Hệ điều hành kiểm thử	Windows, chạy qua PowerShell
Định dạng input	SPMF, mỗi giao dịch là một dòng; các item được phân tách bằng khoảng trắng
Định dạng output	SPMF, dạng <code>itemset #SUP: support</code>
Công cụ kiểm thử	Gói Test của Julia và các script PowerShell <code>.ps1</code>
Script chính	<code>run_tests.ps1</code> , <code>check_algorithms.ps1</code> và <code>run_optimization_check.ps1</code>

Trong phạm vi chương này, các tập dữ liệu kiểm thử là các CSDL toy có kích thước nhỏ. Các tập này được dùng để kiểm tra tính đúng đắn của từng bước cài đặt và đối chiếu với kết quả mong đợi. Việc đánh giá trên các tập benchmark lớn và so sánh với SPMF được trình bày ở Chương thực nghiệm.

3.2 Tổ chức mã nguồn

Mã nguồn được chia thành nhiều file theo chức năng để giúp dễ kiểm thử và bảo trì. Module **PrePostFIM** đóng vai trò gom các thành phần con và export những hàm cần dùng ở bên ngoài. Cách tổ chức này giúp tách rõ bốn nhóm nhiệm vụ: biểu diễn dữ liệu, tiền xử lý, xây dựng cấu trúc PPC-tree/N-list và khai thác frequent itemset.

Bảng 25 mô tả vai trò của các file chính trong project.

Bảng 19: Tổ chức các file mã nguồn chính

File	Vai trò
<code>PrePostFIM.jl</code>	Module chính, include các file thành phần và export các kiểu dữ liệu, hàm đọc/ghi, hàm khai thác và hàm kiểm tra kết quả.
<code>structures/transaction_db.jl</code>	Định nghĩa kiểu <code>TransactionDB</code> để lưu danh sách giao dịch và các hàm thống kê cơ bản như số giao dịch, số item và độ dài giao dịch trung bình.
<code>structures/ppc_tree.jl</code>	Định nghĩa <code>PPCNode</code> , <code>PPCTree</code> ; thực hiện thao tác chèn giao dịch vào cây, gán mã <i>pre/post</i> và thu thập node.

Tiếp tục ở trang sau

Bảng 19 – tiếp theo

File	Vai trò
structures/nlist.jl	Định nghĩa <code>NListEntry</code> , tính support của N-list, tạo N-list cho item đơn và giao hai N-list.
utils/preprocessing.jl	Đếm support của item đơn, lọc item không phổ biến, xác định thứ tự toàn cục và sắp xếp hoặc lọc giao dịch.
utils/io_spmf.jl	Đọc CSDL theo định dạng SPMF và ghi kết quả frequent itemset theo định dạng SPMF.
utils/metrics.jl	Đọc file output, chuẩn hóa kết quả và so sánh hai tập kết quả.
utils/timer.jl	Đo thời gian chạy của một hàm.
algorithm/output.jl	Chuẩn hóa thứ tự itemset và định dạng một dòng kết quả output.
algorithm/mining.jl	Cài đặt thủ tục khai thác đệ quy từ các N-list ứng viên.
algorithm/prepost.jl	Xây dựng pipeline chính của thuật toán, gồm <code>prepost_basic</code> , <code>prepost_optimized</code> và <code>prepost</code> .
src/cli.jl	Giao diện dòng lệnh, nhận tham số input, <i>minsup</i> và output.

Luồng include trong module chính được tổ chức theo thứ tự phụ thuộc: trước hết là các cấu trúc dữ liệu, sau đó là các tiện ích tiền xử lý/đọc ghi, cuối cùng là các file hiện thực thuật toán. Điều này giúp các hàm trong tầng thuật toán có thể sử dụng trực tiếp các kiểu dữ liệu và hàm tiện ích đã được định nghĩa trước.

3.3 Biểu diễn dữ liệu và cấu trúc chính

3.3.1 Cơ sở dữ liệu giao dịch

CSDL giao dịch được biểu diễn bằng kiểu `TransactionDB`. Mỗi giao dịch là một vector các số nguyên, tương ứng với các item trong giao dịch. Toàn bộ CSDL là một vector các giao dịch:

Listing 2: Biểu diễn CSDL giao dịch

```

1 struct TransactionDB
2     transactions::Vector{Vector{Int}}
3 end

```

Cách biểu diễn này phù hợp với dạng dữ liệu giao dịch thường được SPMF sử

dụng, trong đó mỗi dòng biểu diễn một giao dịch và các item được mã hóa bằng số nguyên [8], [9]. Trước khi xây dựng PPC-tree, mỗi giao dịch được lọc bỏ các item không phổ biến, loại bỏ item lặp trong cùng giao dịch và sắp xếp theo thứ tự toàn cục.

3.3.2 PPC-tree

PPC-tree được cài đặt bằng hai kiểu dữ liệu chính là `PPCNode` và `PPCTree`. Mỗi node lưu item, số đếm, con trỏ đến cha, danh sách node con, mã *pre* và mã *post*:

Listing 3: Cấu trúc node trong PPC-tree

```

1 mutable struct PPCNode
2     item::Union{Int,Nothing}
3     count::Int
4     parent::Union{PPCNode,Nothing}
5     children::Vector{PPCNode}
6     pre::Int
7     post::Int
8 end

```

Node gốc có `item = nothing`. Khi chèn một giao dịch đã được sắp xếp, thuật toán đi từ gốc xuống theo thứ tự item trong giao dịch. Nếu node con mang item tương ứng đã tồn tại, trường `count` được tăng lên. Ngược lại, một node mới được tạo và thêm vào danh sách con.

Sau khi toàn bộ giao dịch được chèn, cây được duyệt để gán mã *pre* và *post*. Mã *pre* được gán khi bắt đầu thăm node, còn mã *post* được gán sau khi đã thăm toàn bộ các node con. Cặp mã này cho phép kiểm tra quan hệ tổ tiên–hậu duệ mà không cần duyệt lại đường đi trên cây [5].

3.3.3 N-list

Một phần tử của N-list được biểu diễn bằng bộ ba (*pre*, *post*, *count*):

Listing 4: Biểu diễn phần tử N-list

```

1 struct NListEntry
2     pre::Int
3     post::Int
4     count::Int
5 end

```

Support của một N-list được tính bằng tổng trường `count` của tất cả phần tử trong N-list:

$$\text{sup}(X) = \sum_{e \in \text{NList}(X)} e.\text{count}. \quad (48)$$

Để tạo N-list cho item đơn, chương trình duyệt các node của PPC-tree, gom những node có cùng item và sắp xếp các entry theo mã *pre*. Khi sinh N-list cho itemset mở rộng, chương trình kiểm tra quan hệ tổ tiên-hậu duệ giữa hai N-list. Với hai entry x và y , điều kiện là:

$$x.pre < y.pre \quad \text{và} \quad x.post > y.post. \quad (49)$$

Nếu điều kiện đúng thì node tương ứng với x là tổ tiên của node tương ứng với y . Theo quy ước N-list được sử dụng trong project và ở Chương 2, entry kết quả giữ PP-code của node tổ tiên và nhận count từ node hậu duệ. Nếu nhiều entry hậu duệ ánh xạ vào cùng một PP-code tổ tiên, các giá trị count phải được cộng gộp. Cách tổ chức này phải được giữ nhất quán giữa hàm `join_nlists`, thủ tục khai thác và phần minh họa thủ công [5].

3.4 Quy trình cài đặt thuật toán PrePost

Pipeline chính của thuật toán được cài đặt trong hàm tạo danh sách ứng viên ban đầu và hai hàm khai thác `prepost_basic`, `prepost_optimized`. Các bước xử lý được tóm tắt trong Hình 6.

```

Input SPMF
|
|- Đọc TransactionDB
|- Đếm support item đơn
|- Lọc item có support < minsup
|- Sắp xếp item theo support giảm dần, phá hòa theo mã item tăng dần
|- Lọc và sắp xếp lại từng giao dịch
|- Xây dựng PPC-tree
|- Gán mã pre và post
|- Tạo N-list cho item đơn
|- Khai thác frequent itemset bằng giao N-list
|- Chuẩn hóa và ghi output SPMF

```

Hình 4: Pipeline cài đặt thuật toán PrePost

Các bước tiền xử lý được cài đặt như sau:

1. Quét CSDL để đếm support của từng item đơn. Nếu một item xuất hiện nhiều lần trong cùng một giao dịch thì chỉ được tính một lần.
2. Lọc bỏ các item có support nhỏ hơn *minsup*.
3. Xác định thứ tự toàn cục. Item có support cao hơn đứng trước. Nếu hai item có cùng support, item có mã nhỏ hơn đứng trước.

4. Với từng giao dịch, loại bỏ item không phổ biến, loại bỏ item trùng lặp và sắp xếp theo thứ tự toàn cục.

Sau bước tiền xử lý, các giao dịch được chèn vào PPC-tree. Từ PPC-tree đã gán mã *pre/post*, chương trình xây dựng N-list của các item đơn và khai thác frequent itemset theo chiều sâu, phù hợp với quy trình tổng quát của PrePost [5].

3.5 Phiên bản cơ bản và phiên bản tối ưu

Project duy trì hai phiên bản có cùng pipeline và cùng quy tắc sinh frequent itemset:

- **prepost_basic**: tạo các N-list ứng viên của một bước mở rộng, lưu chúng vào danh sách trung gian, sau đó mới duyệt danh sách để tính support và loại các ứng viên không đạt *minsup*;
- **prepost_optimized**: tính support ngay sau từng phép giao N-list. Ứng viên không đạt *minsup* bị loại trước khi được thêm vào danh sách trung gian hoặc danh sách mở rộng tiếp theo.

Hai phiên bản đều phải loại itemset không phổ biến trước khi tiếp tục đệ quy; nếu không, việc mở rộng các nhánh đó vừa không cần thiết vừa trái với mục tiêu tĩa dựa trên tính phản đơn điệu của support. Vì vậy, khác biệt được đánh giá ở đây không phải là “bản cơ bản giữ lại itemset không phổ biến”, mà là thời điểm kiểm tra support và việc có hay không vật hóa N-list không đạt ngưỡng trong danh sách trung gian.

Đây là một tối ưu cục bộ, tương đối đơn giản nhưng đáng kiểm tra vì nó có thể giảm số đối tượng N-list tạm thời và số ứng viên phải duyệt lại. Nhóm chủ động không kết hợp đồng thời với các kỹ thuật khác để có thể quy thay đổi quan sát được cho đúng một yếu tố cài đặt.

Về mặt kết quả, hai phiên bản phải sinh ra cùng một tập frequent itemset. Do đó, phiên bản cơ bản được dùng như một đối chứng nội bộ để kiểm tra phiên bản tối ưu. Trong mã nguồn, hàm **prepost** mặc định gọi phiên bản tối ưu:

Listing 5: Hàm PrePost mặc định

```

1 prepost(db::TransactionDB, minsup::Int)::Vector{Tuple{Vector{Int
   },Int}} =
2   prepost_optimized(db, minsup)
```

Thủ tục khai thác đệ quy của phiên bản tối ưu có thể mô tả bằng giả mã trong Thuật toán 7.

```

Mine(prefix, suffixCandidates, minsup)
for each candidate (item, nlist) in suffixCandidates do
  supp ← support(nlist)
  if supp < minsup then continue
  newPrefix ← prefix ∪ {item}
  output (newPrefix, supp)
  nextCandidates ← ∅
  for each later candidate (nextItem, nextNList) do
    joined ← join_nlists(nlist, nextNList)
    if support(joined) ≥ minsup then
      add (nextItem, joined) to nextCandidates
  if nextCandidates is not empty then
    Mine(newPrefix, nextCandidates, minsup)

```

Hình 5: Giải mã khai thác đệ quy bằng N-list trong phiên bản tối ưu

Kỹ thuật tĩa sớm không làm thay đổi tập kết quả vì mọi itemset có support nhỏ hơn $minsup$ đều không phổ biến; theo tính phản đơn điệu của support, mọi tập cha của nó cũng không thể phổ biến [1], [2]. Tối ưu chỉ thay đổi thời điểm loại ứng viên và lượng dữ liệu trung gian được lưu.

3.6 Xử lý đầu vào, đầu ra và giao diện dòng lệnh

Chương trình hỗ trợ định dạng giao dịch tương thích với các thuật toán FIM trong SPMF: mỗi dòng là một giao dịch và các item được phân tách bằng khoảng trắng [8], [9]. Riêng bộ đọc của project còn bỏ qua dòng trống và xem phần sau ký tự # là chú thích. Đây là quy ước của trình đọc do nhóm cài đặt, không phải yêu cầu chung cho mọi file dữ liệu SPMF.

Ví dụ một file input hợp lệ:

Listing 6: Ví dụ input theo định dạng SPMF

```

1 1 2 3
2 1 3 4
3 2 3 5
4 1 2 3 5
5 2 3 4

```

Kết quả output được ghi theo định dạng:

Listing 7: Ví dụ output theo định dạng SPMF

```

1 1 #SUP: 3
2 2 #SUP: 4
3 3 #SUP: 5

```

```
4 1 2 #SUP: 2
```

Trước khi ghi output, các kết quả được chuẩn hóa. Mỗi itemset được sắp xếp tăng dần theo mã item. Toàn bộ danh sách kết quả được sắp xếp theo kích thước itemset tăng dần; nếu hai itemset có cùng kích thước thì sắp xếp theo thứ tự từ điển. Việc chuẩn hóa này giúp so sánh kết quả ổn định và không phụ thuộc vào thứ tự sinh ra trong quá trình đệ quy.

Giao diện dòng lệnh được cài đặt trong file `src/cli.jl`. Chương trình nhận ba tham số bắt buộc: đường dẫn input, ngưỡng *minsup* và đường dẫn output. Cú pháp chạy như sau:

Listing 8: Cú pháp chạy chương trình

```
1 julia --project=. src/cli.jl --input <path> --minsup <integer> --  
   output <path>
```

Ví dụ chạy trên CSDL toy cơ sở:

Listing 9: Ví dụ chạy chương trình trên CSDL toy

```
1 julia --project=. src/cli.jl \  
2   --input data/toy/example_basic.txt \  
3   --minsup 2 \  
4   --output outputs/toy/output_basic.out
```

Sau khi chạy, chương trình in ra thông tin input, output, *minsup*, số lượng frequent itemset và thời gian chạy. Điều này giúp thuận tiện cho việc ghi log thực nghiệm.

3.7 Kiểm thử Level 1 và Level 2

3.7.1 Mục tiêu kiểm thử

Mục tiêu kiểm thử là đảm bảo từng thành phần riêng lẻ và toàn bộ pipeline PrePost hoạt động đúng. Các test bao gồm:

- kiểm tra đọc file SPMF;
- kiểm tra đếm support item đơn;
- kiểm tra lọc item không phổ biến và sắp xếp giao dịch;
- kiểm tra xây dựng PPC-tree;
- kiểm tra gán mã *pre* và *post*;
- kiểm tra tạo và giao N-list;

- kiểm tra kết quả frequent itemset cuối cùng;
- kiểm tra chuẩn hóa, ghi và đọc output.

Kết quả chạy unit test mới nhất được ghi nhận như sau:

Listing 10: Kết quả chạy unit test

```

1 Running Julia tests...
2 Starting PrePostFIM test suite...
3 Test Summary: | Pass  Total  Time
4 PrePostFIM    |  103    103  3.9s
5 Finished PrePostFIM test suite.
```

Như vậy, toàn bộ 103 test đều vượt qua.

3.7.2 Bộ dữ liệu kiểm thử Level 2

Để đáp ứng yêu cầu Level 2, nhóm sử dụng năm CSDL toy khác nhau. Hai CSDL đầu là hai ví dụ minh họa tay ở Chương 2. Ba CSDL còn lại được thiết kế để kiểm tra các tình huống khác nhau: dữ liệu thừa, dữ liệu có item không phổ biến và dữ liệu có giao dịch trùng với itemset dài.

Bảng 26 trình bày bộ dữ liệu kiểm thử Level 2.

Bảng 20: Năm CSDL toy dùng cho kiểm thử Level 2

Dataset	Mục đích kiểm thử	<i>minsup</i>	Số FI
example_basic.txt	Ví dụ cơ sở ở Chương 2	2	13
example_special_single_pat h.txt	Trường hợp PPC-tree một nhánh	2	15
example_sparse.txt	Dữ liệu thừa, ít itemset dài	2	5
example_with_infrequent_it ems.txt	Có item không phổ biến cần bị loại	2	6
example_duplicates_long.txt	Có giao dịch trùng và itemset dài	2	15

Mỗi dataset có một file expected output tương ứng trong thư mục `data/toy/expected`. Script `check_algorithms.ps1` chạy chương trình trên năm dataset, chuẩn hóa output rồi so sánh với expected output.

Kết quả kiểm tra thuật toán được tóm tắt trong Bảng 27.

Bảng 21: Kết quả kiểm tra thuật toán trên năm CSDL toy

Dataset	<i>minsup</i>	Số FI sinh ra	Kết quả so sánh
Basic	2	13	Passed
Special single-path	2	15	Passed
Sparse	2	5	Passed
Infrequent-items	2	6	Passed
Duplicates-long	2	15	Passed

Log thực thi của script kiểm tra thuật toán cho thấy cả năm dataset đều khớp expected output:

Listing 11: Kết quả chạy script kiểm tra thuật toán

```

1 [4/5] Comparing results...
2 [PASSED] Basic dataset output matches expected.
3 [PASSED] Special dataset output matches expected.
4 [PASSED] Sparse dataset output matches expected.
5 [PASSED] Infrequent-items dataset output matches expected.
6 [PASSED] Duplicates-long dataset output matches expected.
7
8 =====
9 Algorithm check PASSED
10 =====

```

Tỉ lệ bộ dữ liệu vượt qua phép đối chiếu là:

$$\text{PassRate} = \frac{5}{5} \times 100\% = 100\%. \quad (50)$$

Không nên gọi đại lượng này là *accuracy*, vì đây không phải chỉ số đánh giá mô hình dự đoán. Kết quả cho thấy output của chương trình khớp expected output trên cả năm CSDL toy, bao gồm itemset và support tương ứng.

3.8 Kiểm tra Level 3: so sánh phiên bản cơ bản và tối ưu

Level 3 yêu cầu áp dụng ít nhất một kỹ thuật tối ưu phù hợp với thuật toán và đo lường mức cải thiện so với bản cơ bản. Trong project này, kỹ thuật được dùng là tĩa sớm ứng viên trong quá trình giao N-list. Cụ thể, sau khi sinh N-list cho một ứng viên mở rộng, chương trình tính support ngay. Nếu support nhỏ hơn *minsup*, ứng viên đó không được đưa vào danh sách mở rộng tiếp theo.

Script `run_optimization_check.ps1` so sánh hai hàm `prepost_basic` và `prepost_optimized` trên năm CSDL toy. Với mỗi dataset, script ghi nhận số lượng itemset, biến `match`, thời

gian chạy của bản cơ bản, thời gian chạy của bản tối ưu và speedup.

Kết quả được trình bày trong Bảng 28.

Bảng 22: Kết quả so sánh phiên bản cơ bản và phiên bản tối ưu

Dataset	<i>minsup</i>	Số FI	Match	Basic (s)	Optimized (s)
toy_basic	2	13	true	0.189	0.000
toy_special_single_path	2	15	true	0.081	0.000
toy_sparse	2	5	true	0.000	0.000
toy_with_infrequent_items	2	6	true	0.000	0.000
toy_duplicates_long	2	15	true	0.000	0.000

Toàn bộ năm dataset đều có `match=true`, nghĩa là phiên bản tối ưu sinh cùng tập frequent itemset và support với phiên bản cơ bản. Kết quả này xác nhận tính tương đương đầu ra trên bộ kiểm thử hiện có, nhưng chưa đủ để chứng minh tương đương cho mọi đầu vào; vai trò đó cần được hỗ trợ thêm bởi lập luận thuật toán và các kiểm thử sinh ngẫu nhiên hoặc vét cạn trên CSDL nhỏ.

Các giá trị thời gian bằng 0.000 xuất hiện vì thời gian chạy quá nhỏ so với số chữ số được ghi trong log, không nhất thiết vì đồng hồ không đủ độ phân giải. Do đó, không thể tính hoặc diễn giải speedup hữu ích từ bảng này; giá trị `Inf` chỉ là hệ quả của phép chia cho một số đã bị làm tròn thành 0. Để đo thời gian đáng tin cậy hơn cần:

- chạy một lượt khởi động để loại ảnh hưởng biên dịch JIT;
- lặp mỗi cấu hình nhiều lần và báo cáo median hoặc minimum ổn định;
- đo trên benchmark đủ lớn;
- đồng thời ghi nhận số phép giao N-list, số ứng viên trung gian và số byte cấp phát, vì đây là các đại lượng gắn trực tiếp với tối ưu đang xét.

Vì vậy, trong Chương này chỉ có thể kết luận rằng tối ưu bảo toàn kết quả trên năm CSDL toy. Kết luận về mức cải thiện thời gian hoặc bộ nhớ phải dựa trên Chương thực nghiệm.

3.9 Đáp ứng các mức yêu cầu cài đặt

Bảng 29 tổng hợp mức độ đáp ứng các yêu cầu cài đặt.

Bảng 23: Tổng hợp mức độ đáp ứng các yêu cầu cài đặt

Mức	Yêu cầu	Kết quả thực hiện
Level 1	Cài đặt đúng thuật toán, xuất toàn bộ frequent itemset và support tương ứng.	Đã cài đặt pipeline PrePost đầy đủ từ đọc dữ liệu, tiền xử lý, PPC-tree, N-list đến khai thác FI. Kết quả toy cơ sở sinh 13 FI và toy single-path sinh 15 FI, khớp expected.
Level 2	Kiểm thử tự động trên ít nhất 5 CSDL khác nhau, bao gồm CSDL từ ví dụ tay.	Đã kiểm thử trên 5 CSDL toy: basic, special single-path, sparse, infrequent-items và duplicates-long. Unit test đạt 103/103; algorithm check pass 5/5 dataset.
Level 3	Tối ưu hóa bộ nhớ/tốc độ và đo mức cải thiện so với bản cơ bản.	Đã cài đặt <code>prepost_optimized</code> với tải sớm ngay sau phép giao N-list và xác nhận <code>match=true</code> trên 5/5 dataset. Bảng toy chưa đủ để định lượng speedup; số liệu benchmark và cấp phát bộ nhớ được trình bày ở Chương thực nghiệm.
Level 4	Hỗ trợ đọc input/output định dạng SPMF và tham số dòng lệnh cho <i>minsup</i> , input, output.	Đã có <code>read_spmf</code> , <code>write_spmf_output</code> và CLI nhận <code>-input</code> , <code>-minsup</code> , <code>-output</code> .

3.10 Nhận xét về cài đặt

Cài đặt hiện tại có một số ưu điểm chính. Thứ nhất, mã nguồn được chia module rõ ràng, giúp kiểm thử độc lập từng thành phần. Thứ hai, output được chuẩn hóa trước khi ghi và so sánh, nhờ đó việc kiểm tra kết quả không bị ảnh hưởng bởi thứ tự sinh itemset trong quá trình đệ quy. Thứ ba, project duy trì đồng thời phiên bản cơ bản và phiên bản

tối ưu, cho phép kiểm tra chéo nội bộ trước khi đánh giá trên dữ liệu lớn.

Tuy nhiên, bộ kiểm thử ở Chương này chủ yếu sử dụng dữ liệu toy nên chưa phản ánh hiệu năng trên dữ liệu lớn. Thời gian của lần gọi đầu còn có thể chứa chi phí biên dịch JIT của Julia; vì vậy số đo một lần trên dữ liệu rất nhỏ không phù hợp để kết luận về speedup [12]. Phần đánh giá hiệu năng cần sử dụng benchmark lớn, nhiều mức *minsup*, chạy lặp, thống kê thời gian và cấp phát bộ nhớ, đồng thời đối chiếu kết quả với triển khai tham chiếu trong SPMF [8], [9].

3.11 Khả năng tái lập

Để các kết quả kiểm thử và thực nghiệm có thể được tái lập, báo cáo nên ghi rõ phiên bản Julia thực tế, nội dung `Project.toml/Manifest.toml`, commit mã nguồn, tham số dòng lệnh, cấu hình phần cứng và phiên bản SPMF được dùng để đối chiếu. Các file expected output nên được quản lý cùng mã nguồn và được sinh hoặc kiểm chứng độc lập bằng liệt kê vết cặn trên CSDL toy. Khi đo hiệu năng, cần tách thời gian đọc file khỏi thời gian khai thác nếu mục tiêu là so sánh phần lõi thuật toán.

3.12 Kết luận chương

Chương này đã trình bày quá trình cài đặt thuật toán PrePost bằng Julia. Chương trình bao gồm các thành phần đọc dữ liệu, tiền xử lý, xây dựng PPC-tree, gán mã *pre/post*, tạo N-list, khai thác frequent itemset và xuất kết quả tương thích với quy ước output được sử dụng để đối chiếu SPMF. Các thành phần chức năng của Level 1, Level 2 và Level 4 đã được hiện thực và kiểm thử. Đối với Level 3, project đã có phiên bản tối ưu và đối chứng cơ bản; phần định lượng mức cải thiện được chuyển sang Chương thực nghiệm.

Kết quả kiểm thử ghi nhận 103/103 unit test vượt qua. Script đối chiếu chạy thành công trên năm CSDL toy và toàn bộ output khớp expected output. Phiên bản tối ưu cũng trùng khớp với phiên bản cơ bản trên năm dataset. Các kết quả này cung cấp bằng chứng ban đầu về tính đúng đắn của cài đặt và là cơ sở để chuyển sang Chương thực nghiệm, nơi cần đánh giá thời gian, bộ nhớ và tính tương đương kết quả trên các benchmark lớn hơn.

4 Cài đặt thuật toán PrePost

Chương này trình bày quá trình cài đặt thuật toán PrePost bằng ngôn ngữ Julia [12]. Mục tiêu là hiện thực các thành phần cốt lõi của thuật toán gốc [5]: đọc cơ sở dữ liệu giao dịch, tiền xử lý theo ngưỡng *minsup*, xây dựng PPC-tree, gán mã *pre/post*, tạo N-list, khai thác toàn bộ frequent itemset và xuất kết quả theo định dạng có thể đối chiếu với thư viện SPMF [8], [9].

Việc cài đặt được tổ chức theo bốn mức yêu cầu. Level 1 tập trung vào tính đúng đắn của thuật toán cơ bản. Level 2 bổ sung kiểm thử tự động trên nhiều CSDL nhỏ. Level 3 cài đặt một phiên bản tối ưu và đối chiếu với phiên bản cơ bản. Level 4 hoàn thiện phần đọc/ghi file và giao diện dòng lệnh.

4.1 Môi trường và công cụ

Thuật toán được cài đặt bằng Julia, một ngôn ngữ hướng tới khả năng biểu đạt ở mức cao đồng thời hỗ trợ biên dịch chuyên biệt theo kiểu dữ liệu [12]. Project sử dụng Julia từ phiên bản 1.9 trở lên và có module chính là **PrePostFIM**. Việc chạy chương trình, kiểm thử và đối chiếu kết quả được thực hiện thông qua các script dòng lệnh trên Windows PowerShell.

Môi trường cài đặt được tóm tắt trong Bảng 24.

Bảng 24: Môi trường và công cụ cài đặt

Thành phần	Mô tả
Ngôn ngữ lập trình	Julia
Module chính	PrePostFIM
Hệ điều hành kiểm thử	Windows, chạy qua PowerShell
Định dạng input	SPMF, mỗi giao dịch là một dòng; các item được phân tách bằng khoảng trắng
Định dạng output	SPMF, dạng <code>itemset #SUP: support</code>
Công cụ kiểm thử	Gói Test của Julia và các script PowerShell có phần mở rộng <code>.ps1</code>
Script chính	<code>run_tests.ps1</code> , <code>check_algorithms.ps1</code> và <code>run_optimization_check.ps1</code>

Trong phạm vi chương này, các tập dữ liệu kiểm thử là các CSDL toy có kích thước nhỏ. Các tập này được dùng để kiểm tra tính đúng đắn của từng bước cài đặt và đối chiếu với kết quả mong đợi. Việc đánh giá trên các tập benchmark lớn và so sánh với SPMF được trình bày ở Chương thực nghiệm.

4.2 Tổ chức mã nguồn

Mã nguồn được chia thành nhiều file theo chức năng để giúp dễ kiểm thử và bảo trì. Module **PrePostFIM** đóng vai trò gom các thành phần con và export những hàm cần dùng ở bên ngoài. Cách tổ chức này giúp tách rõ bốn nhóm nhiệm vụ: biểu diễn dữ liệu, tiền xử lý, xây dựng cấu trúc PPC-tree/N-list và khai thác frequent itemset.

Bảng 25 mô tả vai trò của các file chính trong project.

Bảng 25: Tổ chức các file mã nguồn chính

File	Vai trò
PrePostFIM.jl	Module chính, include các file thành phần và export các kiểu dữ liệu, hàm đọc/ghi, hàm khai thác và hàm kiểm tra kết quả.
structures/transaction_db.jl	Định nghĩa kiểu <code>TransactionDB</code> để lưu danh sách giao dịch và các hàm thống kê cơ bản như số giao dịch, số item và độ dài giao dịch trung bình.
structures/ppc_tree.jl	Định nghĩa <code>PPCNode</code> , <code>PPCTree</code> ; thực hiện thao tác chèn giao dịch vào cây, gán mã <i>pre/post</i> và thu thập node.
structures/nlist.jl	Định nghĩa <code>NListEntry</code> , tính support của N-list, tạo N-list cho item đơn và giao hai N-list.
utils/preprocessing.jl	Đếm support của item đơn, lọc item không phổ biến, xác định thứ tự toàn cục và sắp xếp hoặc lọc giao dịch.
utils/io_spmf.jl	Đọc CSDL theo định dạng SPMF và ghi kết quả frequent itemset theo định dạng SPMF.
utils/metrics.jl	Đọc file output, chuẩn hóa kết quả và so sánh hai tập kết quả.
utils/timer.jl	Đo thời gian chạy của một hàm.
algorithm/output.jl	Chuẩn hóa thứ tự itemset và định dạng một dòng kết quả output.
algorithm/mining.jl	Cài đặt thủ tục khai thác đệ quy từ các N-list ứng viên.
algorithm/prepost.jl	Xây dựng pipeline chính của thuật toán, gồm <code>prepost_basic</code> , <code>prepost_optimized</code> và <code>prepost</code> .
src/cli.jl	Giao diện dòng lệnh, nhận tham số input, <i>minsup</i> và output.

Luồng include trong module chính được tổ chức theo thứ tự phụ thuộc: trước hết là các cấu trúc dữ liệu, sau đó là các tiện ích tiền xử lý/đọc ghi, cuối cùng là các file hiện thực thuật toán. Điều này giúp các hàm trong tầng thuật toán có thể sử dụng trực tiếp các kiểu dữ liệu và hàm tiện ích đã được định nghĩa trước.

4.3 Biểu diễn dữ liệu và cấu trúc chính

4.3.1 Cơ sở dữ liệu giao dịch

CSDL giao dịch được biểu diễn bằng kiểu `TransactionDB`. Mỗi giao dịch là một vector các số nguyên, tương ứng với các item trong giao dịch. Toàn bộ CSDL là một vector các giao dịch:

Listing 12: Biểu diễn CSDL giao dịch

```

1 struct TransactionDB
2     transactions::Vector{Vector{Int}}
3 end

```

Cách biểu diễn này phù hợp với dạng dữ liệu giao dịch thường được SPMF sử dụng, trong đó mỗi dòng biểu diễn một giao dịch và các item được mã hóa bằng số nguyên [8], [9]. Trước khi xây dựng PPC-tree, mỗi giao dịch được lọc bỏ các item không phổ biến, loại bỏ item lặp trong cùng giao dịch và sắp xếp theo thứ tự toàn cục.

4.3.2 PPC-tree

PPC-tree được cài đặt bằng hai kiểu dữ liệu chính là `PPCNode` và `PPCTree`. Mỗi node lưu item, số đếm, con trỏ đến cha, danh sách node con, mã *pre* và mã *post*:

Listing 13: Cấu trúc node trong PPC-tree

```

1 mutable struct PPCNode
2     item::Union{Int,Nothing}
3     count::Int
4     parent::Union{PPCNode,Nothing}
5     children::Vector{PPCNode}
6     pre::Int
7     post::Int
8 end

```

Node gốc có `item = nothing`. Khi chèn một giao dịch đã được sắp xếp, thuật toán đi từ gốc xuống theo thứ tự item trong giao dịch. Nếu node con mang item tương ứng đã tồn tại, trường `count` được tăng lên. Ngược lại, một node mới được tạo và thêm vào danh sách con.

Sau khi toàn bộ giao dịch được chèn, cây được duyệt để gán mã *pre* và *post*. Mã *pre* được gán khi bắt đầu thăm node, còn mã *post* được gán sau khi đã thăm toàn bộ các node con. Cặp mã này cho phép kiểm tra quan hệ tổ tiên-hậu duệ mà không cần duyệt lại đường đi trên cây [5].

4.3.3 N-list

Một phần tử của N-list được biểu diễn bằng bộ ba ($pre, post, count$):

Listing 14: Biểu diễn phần tử N-list

```

1 struct NListEntry
2     pre::Int
3     post::Int
4     count::Int
5 end

```

Support của một N-list được tính bằng tổng trường `count` của tất cả phần tử trong N-list:

$$sup(X) = \sum_{e \in NList(X)} e.count. \quad (51)$$

Để tạo N-list cho item đơn, chương trình duyệt các node của PPC-tree, gom những node có cùng item và sắp xếp các entry theo mã *pre*. Khi sinh N-list cho itemset mở rộng, chương trình kiểm tra quan hệ tổ tiên-hậu duệ giữa hai N-list. Với hai entry x và y , điều kiện là:

$$x.pre < y.pre \quad \text{và} \quad x.post > y.post. \quad (52)$$

Nếu điều kiện đúng thì node tương ứng với x là tổ tiên của node tương ứng với y . Theo quy ước N-list được sử dụng trong project và ở Chương 2, entry kết quả giữ PP-code của node tổ tiên và nhận count từ node hậu duệ. Nếu nhiều entry hậu duệ ánh xạ vào cùng một PP-code tổ tiên, các giá trị count phải được cộng gộp. Cách tổ chức này phải được giữ nhất quán giữa hàm `join_nlists`, thủ tục khai thác và phần minh họa thủ công [5].

4.4 Quy trình cài đặt thuật toán PrePost

Pipeline chính của thuật toán được cài đặt trong hàm tạo danh sách ứng viên ban đầu và hai hàm khai thác `prepost_basic`, `prepost_optimized`. Các bước xử lý được tóm tắt trong Hình 6.

```

Input SPMF
|
|- Đọc TransactionDB
|- Đếm support item đơn
|- Lọc item có support < minsup
|- Sắp xếp item theo support giảm dần, phá hòa theo mã item tăng dần
|- Lọc và sắp xếp lại từng giao dịch
|- Xây dựng PPC-tree
|- Gán mã pre và post
|- Tạo N-list cho item đơn
|- Khai thác frequent itemset bằng giao N-list
|- Chuẩn hóa và ghi output SPMF

```

Hình 6: Pipeline cài đặt thuật toán PrePost

Các bước tiền xử lý được cài đặt như sau:

1. Quét CSDL để đếm support của từng item đơn. Nếu một item xuất hiện nhiều lần trong cùng một giao dịch thì chỉ được tính một lần.
2. Lọc bỏ các item có support nhỏ hơn *minsup*.
3. Xác định thứ tự toàn cục. Item có support cao hơn đứng trước. Nếu hai item có cùng support, item có mã nhỏ hơn đứng trước.
4. Với từng giao dịch, loại bỏ item không phổ biến, loại bỏ item trùng lặp và sắp xếp theo thứ tự toàn cục.

Sau bước tiền xử lý, các giao dịch được chèn vào PPC-tree. Từ PPC-tree đã gán mã *pre/post*, chương trình xây dựng N-list của các item đơn và khai thác frequent itemset theo chiều sâu, phù hợp với quy trình tổng quát của PrePost [5].

4.5 Phiên bản cơ bản và phiên bản tối ưu

Project duy trì hai phiên bản có cùng pipeline và cùng quy tắc sinh frequent itemset:

- **prepost_basic**: tạo các N-list ứng viên của một bước mở rộng, lưu chúng vào danh sách trung gian, sau đó mới duyệt danh sách để tính support và loại các ứng viên không đạt *minsup*;
- **prepost_optimized**: tính support ngay sau từng phép giao N-list. Ứng viên không đạt *minsup* bị loại trước khi được thêm vào danh sách trung gian hoặc danh sách mở rộng tiếp theo.

Hai phiên bản đều phải loại itemset không phổ biến trước khi tiếp tục đệ quy; nếu không, việc mở rộng các nhánh đó vừa không cần thiết vừa trái với mục tiêu tĩa dựa trên tính phản đơn điệu của support. Vì vậy, khác biệt được đánh giá ở đây không phải là “bản cơ bản giữ lại itemset không phổ biến”, mà là thời điểm kiểm tra support và việc có hay không vật hóa N-list không đạt ngưỡng trong danh sách trung gian.

Đây là một tối ưu cục bộ, tương đối đơn giản nhưng đáng kiểm tra vì nó có thể giảm số đối tượng N-list tạm thời và số ứng viên phải duyệt lại. Nhóm chủ động không kết hợp đồng thời với các kỹ thuật khác để có thể quy thay đổi quan sát được cho đúng một yếu tố cài đặt.

Về mặt kết quả, hai phiên bản phải sinh ra cùng một tập frequent itemset. Do đó, phiên bản cơ bản được dùng như một đối chứng nội bộ để kiểm tra phiên bản tối ưu. Trong mã nguồn, hàm `prepost` mặc định gọi phiên bản tối ưu:

Listing 15: Hàm PrePost mặc định

```

1 prepost(db::TransactionDB, minsup::Int)::Vector{Tuple{Vector{Int
  },Int}} =
2   prepost_optimized(db, minsup)
```

Thủ tục khai thác đệ quy của phiên bản tối ưu có thể mô tả bằng giả mã trong Thuật toán 7.

```

Mine(prefix, suffixCandidates, minsup)
for each candidate (item, nlist) in suffixCandidates do
  supp ← support(nlist)
  if supp < minsup then continue
  newPrefix ← prefix ∪ {item}
  output (newPrefix, supp)
  nextCandidates ← ∅
  for each later candidate (nextItem, nextNList) do
    joined ← join_nlists(nlist, nextNList)
    if support(joined) ≥ minsup then
      add (nextItem, joined) to nextCandidates
    if nextCandidates is not empty then
      Mine(newPrefix, nextCandidates, minsup)
```

Hình 7: Giả mã khai thác đệ quy bằng N-list trong phiên bản tối ưu

Kỹ thuật tĩa sớm không làm thay đổi tập kết quả vì mọi itemset có support nhỏ hơn *minsup* đều không phổ biến; theo tính phản đơn điệu của support, mọi tập cha của nó cũng không thể phổ biến [1], [2]. Tối ưu chỉ thay đổi thời điểm loại ứng viên và lượng dữ liệu trung gian được lưu.

4.6 Xử lý đầu vào, đầu ra và giao diện dòng lệnh

Chương trình hỗ trợ định dạng giao dịch tương thích với các thuật toán FIM trong SPMF: mỗi dòng là một giao dịch và các item được phân tách bằng khoảng trắng [8], [9]. Riêng bộ đọc của project còn bỏ qua dòng trống và xem phần sau ký tự # là chú thích. Đây là quy ước của trình đọc do nhóm cài đặt, không phải yêu cầu chung cho mọi file dữ liệu SPMF.

Ví dụ một file input hợp lệ:

Listing 16: Ví dụ input theo định dạng SPMF

```
1 1 2 3
2 1 3 4
3 2 3 5
4 1 2 3 5
5 2 3 4
```

Kết quả output được ghi theo định dạng:

Listing 17: Ví dụ output theo định dạng SPMF

```
1 1 #SUP: 3
2 2 #SUP: 4
3 3 #SUP: 5
4 1 2 #SUP: 2
```

Trước khi ghi output, các kết quả được chuẩn hóa. Mỗi itemset được sắp xếp tăng dần theo mã item. Toàn bộ danh sách kết quả được sắp xếp theo kích thước itemset tăng dần; nếu hai itemset có cùng kích thước thì sắp xếp theo thứ tự từ điển. Việc chuẩn hóa này giúp so sánh kết quả ổn định và không phụ thuộc vào thứ tự sinh ra trong quá trình đệ quy.

Giao diện dòng lệnh được cài đặt trong file `src/cli.jl`. Chương trình nhận ba tham số bắt buộc: đường dẫn input, ngưỡng *minsup* và đường dẫn output. Cú pháp chạy như sau:

Listing 18: Cú pháp chạy chương trình

```
1 julia --project=. src/cli.jl --input <path> --minsup <integer> --
  output <path>
```

Ví dụ chạy trên CSDL toy cơ sở:

Listing 19: Ví dụ chạy chương trình trên CSDL toy

```
1 julia --project=. src/cli.jl \
2   --input data/toy/example_basic.txt \
3   --minsup 2 \
```



```
4 --output outputs/toy/output_basic.out
```

Sau khi chạy, chương trình in ra thông tin input, output, *minsup*, số lượng frequent itemset và thời gian chạy. Điều này giúp thuận tiện cho việc ghi log thực nghiệm.

4.7 Kiểm thử Level 1 và Level 2

4.7.1 Mục tiêu kiểm thử

Mục tiêu kiểm thử là đảm bảo từng thành phần riêng lẻ và toàn bộ pipeline PrePost hoạt động đúng. Các test bao gồm:

- kiểm tra đọc file SPMF;
- kiểm tra đếm support item đơn;
- kiểm tra lọc item không phổ biến và sắp xếp giao dịch;
- kiểm tra xây dựng PPC-tree;
- kiểm tra gán mã *pre* và *post*;
- kiểm tra tạo và giao N-list;
- kiểm tra kết quả frequent itemset cuối cùng;
- kiểm tra chuẩn hóa, ghi và đọc output.

Kết quả chạy unit test mới nhất được ghi nhận như sau:

Listing 20: Kết quả chạy unit test

```
1 Running Julia tests...
2 Starting PrePostFIM test suite...
3 Test Summary: | Pass  Total  Time
4 PrePostFIM    |  103    103   3.9s
5 Finished PrePostFIM test suite.
```

Như vậy, toàn bộ 103 test đều vượt qua.

4.7.2 Bộ dữ liệu kiểm thử Level 2

Để đáp ứng yêu cầu Level 2, nhóm sử dụng năm CSDL toy khác nhau. Hai CSDL đầu là hai ví dụ minh họa tay ở Chương 2. Ba CSDL còn lại được thiết kế để kiểm tra các tình huống khác nhau: dữ liệu thừa, dữ liệu có item không phổ biến và dữ liệu có giao dịch trùng với itemset dài.

Bảng 26 trình bày bộ dữ liệu kiểm thử Level 2.

Bảng 26: Năm CSDL toy dùng cho kiểm thử Level 2

Dataset	Mục đích kiểm thử	<i>minsup</i>	Số FI
example_basic.txt	Ví dụ cơ sở ở Chương 2	2	13
example_special_single_pat h.txt	Trường hợp PPC-tree một nhánh	2	15
example_sparse.txt	Dữ liệu thưa, ít itemset dài	2	5
example_with_infrequent_it ems.txt	Có item không phổ biến cần bị loại	2	6
example_duplicates_long.txt	Có giao dịch trùng và itemset dài	2	15

Mỗi dataset có một file expected output tương ứng trong thư mục `data/toy/expected`. Script `check_algorithms.ps1` chạy chương trình trên năm dataset, chuẩn hóa output rồi so sánh với expected output.

Kết quả kiểm tra thuật toán được tóm tắt trong Bảng 27.

Bảng 27: Kết quả kiểm tra thuật toán trên năm CSDL toy

Dataset	<i>minsup</i>	Số FI sinh ra	Kết quả so sánh
Basic	2	13	Passed
Special single-path	2	15	Passed
Sparse	2	5	Passed
Infrequent-items	2	6	Passed
Duplicates-long	2	15	Passed

Log thực thi của script kiểm tra thuật toán cho thấy cả năm dataset đều khớp expected output:

Listing 21: Kết quả chạy script kiểm tra thuật toán

```

1 [4/5] Comparing results...
2 [PASSED] Basic dataset output matches expected.
3 [PASSED] Special dataset output matches expected.
4 [PASSED] Sparse dataset output matches expected.
5 [PASSED] Infrequent-items dataset output matches expected.
6 [PASSED] Duplicates-long dataset output matches expected.
7
8 =====
9 Algorithm check PASSED
10 =====

```

Tỉ lệ bộ dữ liệu vượt qua phép đối chiếu là:

$$\text{PassRate} = \frac{5}{5} \times 100\% = 100\%. \quad (53)$$

Không nên gọi đại lượng này là *accuracy*, vì đây không phải chỉ số đánh giá mô hình dự đoán. Kết quả cho thấy output của chương trình khớp expected output trên cả năm CSDL toy, bao gồm itemset và support tương ứng.

4.8 Kiểm tra Level 3: so sánh phiên bản cơ bản và tối ưu

Level 3 yêu cầu áp dụng ít nhất một kỹ thuật tối ưu phù hợp với thuật toán và đo lường mức cải thiện so với bản cơ bản. Trong project này, kỹ thuật được dùng là tĩa sớm ứng viên trong quá trình giao N-list. Cụ thể, sau khi sinh N-list cho một ứng viên mở rộng, chương trình tính support ngay. Nếu support nhỏ hơn *minsup*, ứng viên đó không được đưa vào danh sách mở rộng tiếp theo.

Script `run_optimization_check.ps1` được dùng để so sánh hai hàm `prepost_basic` và `prepost_optimized` trên năm CSDL toy. Với mỗi dataset, script ghi nhận số lượng itemset, trạng thái `match`, thời gian chạy của hai phiên bản và hệ số `speedup`.

Kết quả được trình bày trong Bảng 28.

Bảng 28: Kết quả so sánh phiên bản cơ bản và phiên bản tối ưu

Dataset	<i>minsup</i>	Số FI	Match	Basic (s)	Optimized (s)
<code>toy_basic</code>	2	13	true	0.189	0.000
<code>toy_special_single_path</code>	2	15	true	0.081	0.000
<code>toy_sparse</code>	2	5	true	0.000	0.000
<code>toy_with_infrequent_items</code>	2	6	true	0.000	0.000
<code>toy_duplicates_long</code>	2	15	true	0.000	0.000

Toàn bộ năm dataset đều có `match=true`, nghĩa là phiên bản tối ưu sinh cùng tập frequent itemset và support với phiên bản cơ bản. Kết quả này xác nhận tính tương đương đầu ra trên bộ kiểm thử hiện có, nhưng chưa đủ để chứng minh tương đương cho mọi đầu vào; vai trò đó cần được hỗ trợ thêm bởi lập luận thuật toán và các kiểm thử sinh ngẫu nhiên hoặc vét cạn trên CSDL nhỏ.

Các giá trị thời gian bằng 0.000 xuất hiện vì thời gian chạy quá nhỏ so với số chữ số được ghi trong log, không nhất thiết vì đồng hồ không đủ độ phân giải. Do đó, không thể tính hoặc diễn giải `speedup` hữu ích từ bảng này; giá trị `Inf` chỉ là hệ quả của phép chia cho một số đã bị làm tròn thành 0. Để đo thời gian đáng tin cậy hơn cần:

- chạy một lượt khởi động để loại ảnh hưởng biên dịch JIT;

- lập mỗi cấu hình nhiều lần và báo cáo median hoặc minimum ổn định;
- đo trên benchmark đủ lớn;
- đồng thời ghi nhận số phép giao N-list, số ứng viên trung gian và số byte cấp phát, vì đây là các đại lượng gắn trực tiếp với tối ưu đang xét.

Vì vậy, trong Chương này chỉ có thể kết luận rằng tối ưu bảo toàn kết quả trên năm CSDL toy. Kết luận về mức cải thiện thời gian hoặc bộ nhớ phải dựa trên Chương thực nghiệm.

4.9 Đáp ứng các mức yêu cầu cài đặt

Bảng 29 tổng hợp mức độ đáp ứng các yêu cầu cài đặt.

Bảng 29: Tổng hợp mức độ đáp ứng các yêu cầu cài đặt

Mức	Yêu cầu	Kết quả thực hiện
Level 1	Cài đặt đúng thuật toán, xuất toàn bộ frequent itemset và support tương ứng.	Đã cài đặt pipeline PrePost đầy đủ từ đọc dữ liệu, tiền xử lý, PPC-tree, N-list đến khai thác FI. Kết quả toy cơ sở sinh 13 FI và toy single-path sinh 15 FI, khớp expected.
Level 2	Kiểm thử tự động trên ít nhất 5 CSDL khác nhau, bao gồm CSDL từ ví dụ tay.	Đã kiểm thử trên 5 CSDL toy: basic, special single-path, sparse, infrequent-items và duplicates-long. Unit test đạt 103/103; algorithm check pass 5/5 dataset.

Mức	Yêu cầu	Kết quả thực hiện
Level 3	Tối ưu hóa bộ nhớ/tốc độ và đo mức cải thiện so với bản cơ bản.	Đã cài đặt <code>prepost_optimized</code> với tải sớm ngay sau phép giao N-list và xác nhận <code>match=true</code> trên 5/5 dataset. Bảng toy chưa đủ để định lượng speedup; số liệu benchmark và cấp phát bộ nhớ được trình bày ở Chương thực nghiệm.
Level 4	Hỗ trợ đọc input/output định dạng SPMF và tham số dòng lệnh cho <code>minsup</code> , input, output.	Đã có <code>read_spmf</code> , <code>write_spmf_output</code> và CLI nhận <code>-input</code> , <code>-minsup</code> , <code>-output</code> .

4.10 Nhận xét về cài đặt

Cài đặt hiện tại có một số ưu điểm chính. Thứ nhất, mã nguồn được chia module rõ ràng, giúp kiểm thử độc lập từng thành phần. Thứ hai, output được chuẩn hóa trước khi ghi và so sánh, nhờ đó việc kiểm tra kết quả không bị ảnh hưởng bởi thứ tự sinh itemset trong quá trình đệ quy. Thứ ba, project duy trì đồng thời phiên bản cơ bản và phiên bản tối ưu, cho phép kiểm tra chéo nội bộ trước khi đánh giá trên dữ liệu lớn.

Tuy nhiên, bộ kiểm thử ở Chương này chủ yếu sử dụng dữ liệu toy nên chưa phản ánh hiệu năng trên dữ liệu lớn. Thời gian của lần gọi đầu còn có thể chứa chi phí biên dịch JIT của Julia; vì vậy số đo một lần trên dữ liệu rất nhỏ không phù hợp để kết luận về speedup [12]. Phần đánh giá hiệu năng cần sử dụng benchmark lớn, nhiều mức `minsup`, chạy lặp, thống kê thời gian và cấp phát bộ nhớ, đồng thời đối chiếu kết quả với triển khai tham chiếu trong SPMF [8], [9].

4.11 Khả năng tái lập

Để các kết quả kiểm thử và thực nghiệm có thể được tái lập, báo cáo nên ghi rõ phiên bản Julia thực tế, nội dung `Project.toml/Manifest.toml`, commit mã nguồn, tham số dòng lệnh, cấu hình phần cứng và phiên bản SPMF được dùng để đối chiếu. Các file expected output nên được quản lý cùng mã nguồn và được sinh hoặc kiểm chứng độc lập bằng liệt kê vết cặn trên CSDL toy. Khi đo hiệu năng, cần tách thời gian đọc file khỏi thời gian khai thác nếu mục tiêu là so sánh phần lõi thuật toán.

4.12 Kết luận chương

Chương này đã trình bày quá trình cài đặt thuật toán PrePost bằng Julia. Chương trình bao gồm các thành phần đọc dữ liệu, tiền xử lý, xây dựng PPC-tree, gán mã *pre/post*, tạo N-list, khai thác frequent itemset và xuất kết quả tương thích với quy ước output được sử dụng để đối chiếu SPMF. Các thành phần chức năng của Level 1, Level 2 và Level 4 đã được hiện thực và kiểm thử. Đối với Level 3, project đã có phiên bản tối ưu và đối chứng cơ bản; phần định lượng mức cải thiện được chuyển sang Chương thực nghiệm.

Kết quả kiểm thử ghi nhận 103/103 unit test vượt qua. Script đối chiếu chạy thành công trên năm CSDL toy và toàn bộ output khớp expected output. Phiên bản tối ưu cũng trùng khớp với phiên bản cơ bản trên năm dataset. Các kết quả này cung cấp bằng chứng ban đầu về tính đúng đắn của cài đặt và là cơ sở để chuyển sang Chương thực nghiệm, nơi cần đánh giá thời gian, bộ nhớ và tính tương đương kết quả trên các benchmark lớn hơn.

5 Ứng dụng thực tế: Phân tích giỏ hàng

Chương này minh họa việc ứng dụng khai thác tập mục phổ biến vào bài toán phân tích giỏ hàng. Từ các frequent itemset được khai thác bằng PrePost, nhóm sinh luật kết hợp nhằm phát hiện những nhóm sản phẩm thường xuất hiện cùng nhau trong giao dịch [1], [2]. Kết quả được dùng để đề xuất các hướng ứng dụng như bố trí sản phẩm, thiết kế chương trình mua kèm và xây dựng gợi ý tại điểm bán.

Dữ liệu sử dụng là `groceries.txt`, gồm 9,835 giao dịch và 169 item. Đây là bộ dữ liệu *Groceries* thường được sử dụng trong các ví dụ khai phá luật kết hợp của hệ sinh thái `arules` [13]. Trong file của project, mỗi dòng biểu diễn một giao dịch và các item được phân tách bằng dấu phẩy. Toàn bộ quá trình tiền xử lý, khai thác, sinh luật và trực quan hóa được thực hiện trong notebook ứng dụng; các hình được xuất sang thư mục `figures/` để chèn vào báo cáo.

5.1 Mục tiêu ứng dụng

Trong ngữ cảnh bán lẻ, mỗi giao dịch được biểu diễn như một tập item. Khai thác frequent itemset giúp phát hiện các nhóm sản phẩm có support không nhỏ hơn một ngưỡng cho trước. Từ một frequent itemset Z , có thể sinh luật:

$$X \Rightarrow Y, \quad X \cup Y = Z, \quad X \cap Y = \emptyset, \quad (54)$$

trong đó X là vế trái (*antecedent*) và Y là vế phải (*consequent*) [1], [2].

Các chỉ số được sử dụng trong chương gồm:

$$\text{support}(X \Rightarrow Y) = \text{support}(X \cup Y), \quad (55)$$

$$\text{confidence}(X \Rightarrow Y) = \frac{\text{support}(X \cup Y)}{\text{support}(X)}, \quad (56)$$

$$\text{lift}(X \Rightarrow Y) = \frac{\text{confidence}(X \Rightarrow Y)}{\text{support}(Y)}, \quad (57)$$

$$\text{leverage}(X \Rightarrow Y) = \text{support}(X \cup Y) - \text{support}(X) \text{support}(Y), \quad (58)$$

$$\text{conviction}(X \Rightarrow Y) = \frac{1 - \text{support}(Y)}{1 - \text{confidence}(X \Rightarrow Y)}. \quad (59)$$

Support và confidence là hai chỉ số nền tảng của luật kết hợp. Lift, leverage và conviction bổ sung góc nhìn về mức độ lệch khỏi giả thuyết độc lập giữa hai vế của luật [13], [14]. Cụ thể, lift lớn hơn 1 cho thấy X và Y đồng xuất hiện nhiều hơn mức kỳ vọng nếu hai biến cố độc lập; lift bằng 1 tương ứng với mức độc lập theo chỉ số này; lift nhỏ hơn 1 gợi ý quan hệ âm.

Một luật như **butter** $\Rightarrow$ **whole milk** không được hiểu là việc mua bơ gây ra việc mua sữa. Luật chỉ cho biết trong dữ liệu quan sát, tần suất xuất hiện của **whole milk** trong nhóm giao dịch có **butter** cao đến mức nào so với toàn bộ dữ liệu. Các luật có thể hỗ trợ:

- lựa chọn vị trí trưng bày hoặc vùng gợi ý mua kèm;
- thiết kế thử nghiệm combo hoặc khuyến mãi có điều kiện;
- tạo candidate cho hệ thống gợi ý tại quầy thanh toán;
- hình thành giả thuyết về hành vi mua hàng để kiểm thử tiếp.

Trong chương này, luật không được xếp hạng chỉ bằng lift. Nhóm xem xét đồng thời support count, support, confidence, lift và bối cảnh sản phẩm để hạn chế việc chọn các luật mạnh về tỉ lệ nhưng xuất hiện quá ít.

5.2 Mô tả dữ liệu và tiền xử lý

Notebook đọc mỗi dòng như một giao dịch, tách item theo dấu phẩy, loại bỏ khoảng trắng ở đầu và cuối chuỗi, rồi chuyển tên item về chữ thường. Các item lặp trong cùng giao dịch được loại bỏ vì support của itemset được tính theo số giao dịch chứa itemset, không theo số lần item lặp lại trong một giao dịch.

Bảng 30 trình bày thông tin tổng quan của dữ liệu sau tiền xử lý.

Bảng 30: Tổng quan dữ liệu `groceries.txt` sau tiền xử lý

Thông tin	Giá trị
Số dòng không rỗng ban đầu	9,835
Số giao dịch sau xử lý	9,835
Số item khác nhau	169
Độ dài giỏ hàng trung bình	4.41 item
Độ dài giỏ hàng nhỏ nhất	1 item
Độ dài giỏ hàng lớn nhất	32 item

Bộ dữ liệu có quy mô phù hợp cho một nghiên cứu minh họa. Tuy nhiên, số giao dịch lớn không tự động bảo đảm mọi luật đều đáng tin cậy: độ ổn định còn phụ thuộc support count của từng luật và quy trình lựa chọn ngưỡng. Độ dài giỏ hàng trung bình là 4.41 item; vì vậy, các luật có một hoặc hai item ở vế trái thường dễ diễn giải hơn, nhưng đây là lựa chọn về khả năng sử dụng chứ không phải điều kiện của thuật toán.

5.3 Khám phá dữ liệu ban đầu

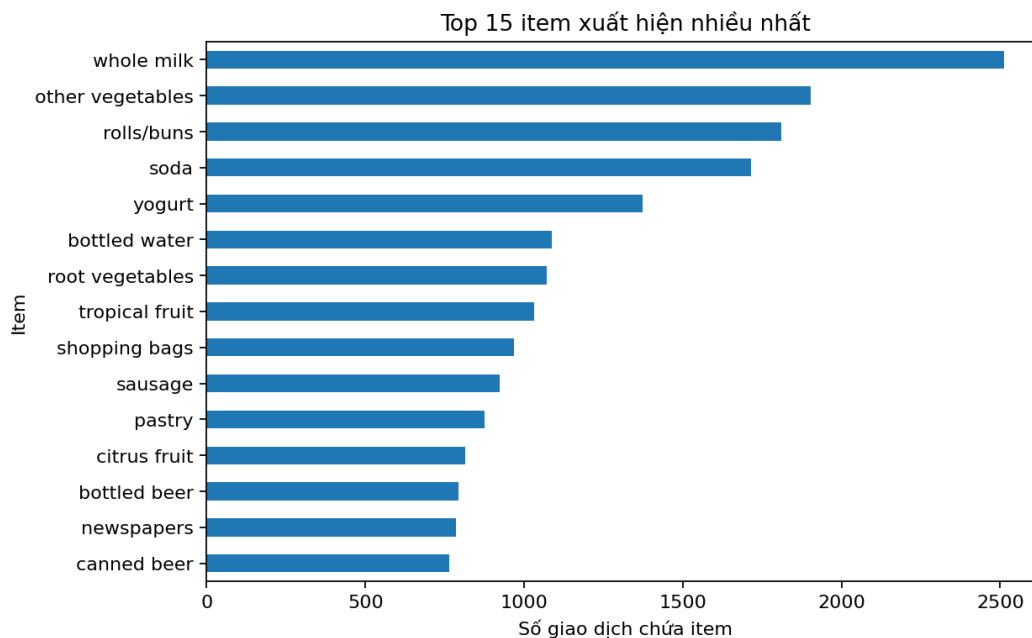
Trước khi chạy thuật toán khai thác frequent itemset, nhóm thực hiện một số bước khám phá dữ liệu nhằm hiểu đặc điểm của tập giao dịch. Các phân tích này gồm thống kê top item phổ biến, ma trận đồng xuất hiện giữa các item phổ biến và phân bố kích thước giỏ hàng.

5.3.1 Các item phổ biến nhất

Bảng 31 liệt kê 15 item có số lượng giao dịch chứa item cao nhất. Đây là các sản phẩm nền tảng có tần suất xuất hiện lớn và thường đóng vai trò quan trọng khi hình thành luật kết hợp.

Bảng 31: Top 15 item xuất hiện nhiều nhất trong dữ liệu

Item	Số giao dịch	Support
whole milk	2,513	0.2555
other vegetables	1,903	0.1935
rolls/buns	1,809	0.1839
soda	1,715	0.1744
yogurt	1,372	0.1395
bottled water	1,087	0.1105
root vegetables	1,072	0.1090
tropical fruit	1,032	0.1049
shopping bags	969	0.0985
sausage	924	0.0940
pastry	875	0.0890
citrus fruit	814	0.0828
bottled beer	792	0.0805
newspapers	785	0.0798
canned beer	764	0.0777



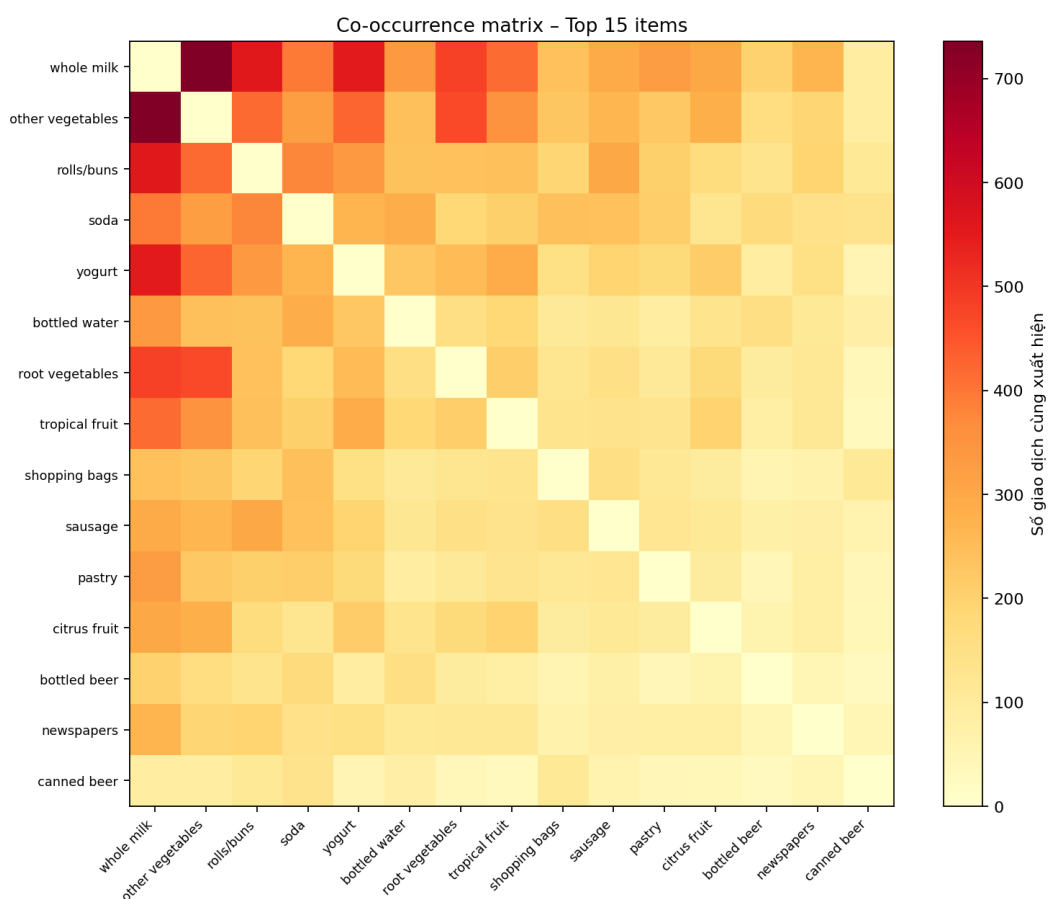
Hình 8: Top 15 item xuất hiện nhiều nhất trong tập groceries

Các item phổ biến nhất là **whole milk**, **other vegetables**, **rolls/buns**, **soda** và **yogurt**. Tuy nhiên, consequent phổ biến có thể làm confidence cao ngay cả khi quan hệ với antecedent không mạnh. Vì vậy, confidence cần được đọc cùng lift và support count

[15].

5.3.2 Ma trận đồng xuất hiện

Để quan sát trực quan mối quan hệ giữa các item phổ biến, nhóm xây dựng ma trận đồng xuất hiện cho 15 item đứng đầu. Mỗi ô trong ma trận cho biết số giao dịch chứa đồng thời hai item tương ứng.



Hình 9: Ma trận đồng xuất hiện của top 15 item phổ biến

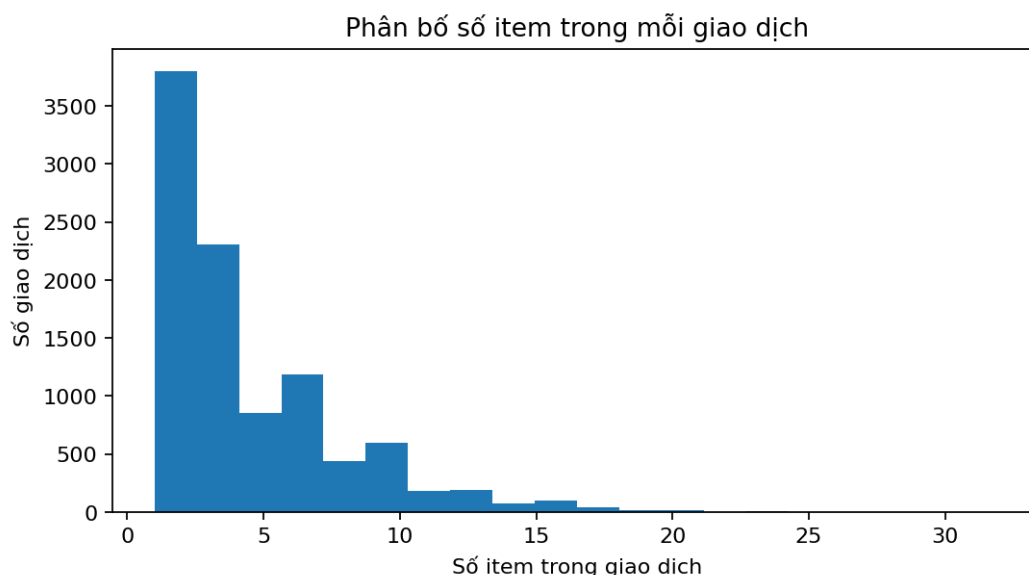
Ma trận đồng xuất hiện cho thấy một số cặp sản phẩm có tần suất cùng xuất hiện cao, đặc biệt quanh các nhóm thực phẩm thiết yếu như sữa, rau củ, bánh mì và sữa chua. Tuy nhiên, ma trận này chỉ phản ánh số lần xuất hiện chung, chưa chuẩn hóa theo mức phổ biến riêng của từng item. Do đó, nó được dùng như bước kiểm tra trực quan ban đầu, còn quyết định chọn luật cần dựa trên các chỉ số như confidence và lift.

5.3.3 Phân bố kích thước giỏ hàng

Phân bố số item trong mỗi giao dịch được trình bày ở Hình 10. Các thống kê mô tả tương ứng được trình bày trong Bảng 32.

Bảng 32: Thống kê kích thước giỏ hàng

Thống kê	Giá trị
Số giao dịch	9,835
Trung bình	4.4095
Độ lệch chuẩn	3.5894
Nhỏ nhất	1
Tứ phân vị 25%	2
Trung vị	3
Tứ phân vị 75%	6
Lớn nhất	32



Hình 10: Phân bố số item trong mỗi giao dịch

Kết quả cho thấy phần lớn giao dịch có số item tương đối nhỏ. Điều này giúp việc diễn giải các luật ngăn trở nên phù hợp hơn. Những giỏ hàng dài vẫn có thể chứa nhiều pattern thú vị, nhưng nếu luật chỉ xuất hiện trong các trường hợp hiếm thì khả năng triển khai rộng trong cửa hàng sẽ thấp.

5.4 Thiết lập khai thác frequent itemset và sinh luật kết hợp

Trong notebook, mỗi item được ánh xạ sang một mã số nguyên trước khi gọi cài đặt `prepost_optimized` qua Julia CLI. Kết quả frequent itemset sau đó được ánh xạ ngược về tên sản phẩm để sinh luật kết hợp. Notebook có thể chứa cơ chế fallback phục vụ kiểm tra kỹ thuật, nhưng toàn bộ bảng và hình trong báo cáo này được tạo từ **Project**

PrePost/optimized qua Julia CLI. Việc ghi rõ backend thực tế là cần thiết để bảo đảm khả năng tái lập.

Các ngưỡng thử nghiệm gồm:

- $\text{minsup} \in \{0.020, 0.015, 0.010, 0.008, 0.005, 0.003\}$;
- $\text{minconf} \in \{0.50, 0.40, 0.30, 0.25, 0.20, 0.15\}$.

Bảng 33 trình bày số lượng luật kết hợp sinh được theo từng cặp ngưỡng.

Bảng 33: Số lượng association rules theo từng cặp minsup và minconf

$\text{minsup} \setminus \text{minconf}$	0.15	0.20	0.25	0.30	0.40	0.50
0.003	3,228	2,368	1,832	1,390	825	422
0.005	1,220	893	673	487	270	120
0.008	514	365	282	205	101	30
0.010	322	234	170	125	62	15
0.015	149	116	78	59	25	2
0.020	91	73	49	37	15	1

Nhóm chọn $\text{minsup} = 0.020$ và $\text{minconf} = 0.40$. Với 9,835 giao dịch, ngưỡng support tuyệt đối được tính bằng:

$$\lceil 0.020 \times 9835 \rceil = 197. \quad (60)$$

Cấu hình này tạo 15 luật, đủ nhỏ để phân tích thủ công nhưng vẫn yêu cầu mỗi luật được hỗ trợ bởi ít nhất 197 giao dịch. Việc chọn ngưỡng mang tính thực nghiệm và phục vụ mục tiêu minh họa; nó không chứng minh đây là cấu hình tối ưu cho kinh doanh. Khi giảm ngưỡng, số luật tăng nhanh và cần thêm bước kiểm soát nhiễu, độ ổn định hoặc kiểm định trên dữ liệu khác.

Bảng 34 tóm tắt cấu hình cuối cùng.

Bảng 34: Cấu hình ngưỡng được chọn cho phần ứng dụng

Thuật toán thực tế được sử dụng	minsup	$\text{minsup}_{\text{abs}}$	minconf	Frequent itemsets	Association rules
Project	0.020	197	0.40	122	15
PrePost/optimized qua Julia CLI					

5.5 Kết quả frequent itemset tiêu biểu

Với cấu hình ngưỡng đã chọn, thuật toán sinh được 122 frequent itemset. Bảng 35 trình bày một số frequent itemset tiêu biểu, bao gồm các item đơn phổ biến nhất, một số cặp item và một số bộ ba item có support đạt ngưỡng.

Bảng 35: Một số frequent itemset tiêu biểu với minsup = 0.020

Itemset	Kích thước	Support count	Support
whole milk	1	2,513	0.2555
other vegetables	1	1,903	0.1935
rolls/buns	1	1,809	0.1839
soda	1	1,715	0.1744
yogurt	1	1,372	0.1395
bottled water	1	1,087	0.1105
root vegetables	1	1,072	0.1090
tropical fruit	1	1,032	0.1049
shopping bags	1	969	0.0985
sausage	1	924	0.0940
other vegetables, whole milk	2	736	0.0748
rolls/buns, whole milk	2	557	0.0566
whole milk, yogurt	2	551	0.0560
root vegetables, whole milk	2	481	0.0489
other vegetables, root vegetables	2	466	0.0474
other vegetables, yogurt	2	427	0.0434
other vegetables, rolls/buns	2	419	0.0426
tropical fruit, whole milk	2	416	0.0423
soda, whole milk	2	394	0.0401
rolls/buns, soda	2	377	0.0383
other vegetables, root vegetables, whole milk	3	228	0.0232
other vegetables, whole milk, yogurt	3	219	0.0223

Các frequent itemset phổ biến nhất chủ yếu xoay quanh những sản phẩm thiết yếu như whole milk, other vegetables, rolls/buns, yogurt và root vegetables. Điều này phù hợp với kết quả khám phá dữ liệu ban đầu. Đáng chú ý, một số bộ ba như other vegetables, root vegetables, whole milk và other vegetables, whole milk, yogurt vẫn đạt support trên 2%, cho thấy có các nhóm sản phẩm thường được mua cùng nhau chứ không chỉ các cặp item đơn giản.

5.6 Kết quả luật kết hợp

Từ các frequent itemset đã khai thác, nhóm sinh association rules và sắp xếp theo lift. Bảng 36 trình bày top 10 luật có lift cao nhất trong số các luật thỏa $\text{minsup} = 0.020$ và $\text{minconf} = 0.40$.

Bảng 36: Top 10 luật kết hợp theo lift

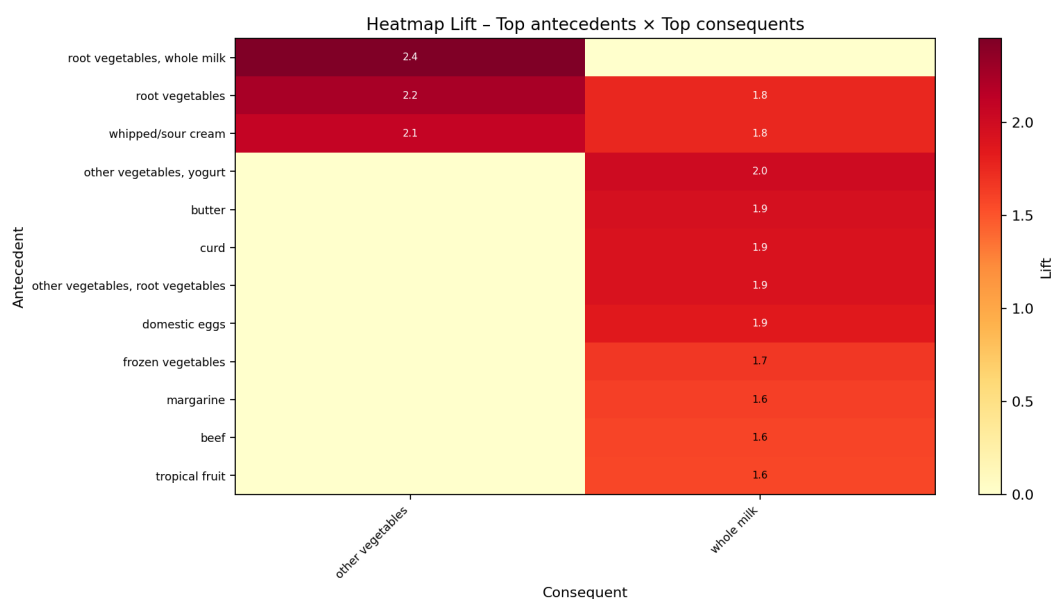
Antecedent	Consequent	Support	Count	Confidence	Lift	Conviction
root vegetables, whole milk	other vegetables	0.0232	228	0.4740	2.4498	1.5333
root vegetables	other vegetables	0.0474	466	0.4347	2.2466	1.4267
whipped/sour cream	other vegetables	0.0289	284	0.4028	2.0819	1.3506
other vegetables, yogurt	whole milk	0.0223	219	0.5129	2.0072	1.5283
butter	whole milk	0.0276	271	0.4972	1.9461	1.4808
curd	whole milk	0.0261	257	0.4905	1.9195	1.4611
other vegetables, root vegetables	whole milk	0.0232	228	0.4893	1.9148	1.4577
domestic eggs	whole milk	0.0300	295	0.4728	1.8502	1.4120
whipped/sour cream	whole milk	0.0322	317	0.4496	1.7598	1.3527
root vegetables	whole milk	0.0489	481	0.4487	1.7560	1.3504

Luật có lift cao nhất là **root vegetables, whole milk $\Rightarrow$ other vegetables**, với confidence 0.4740 và lift 2.4498. Nghĩa là 47.40% giao dịch chứa cả hai item ở vế trái cũng chứa **other vegetables**. Lift 2.4498 cho biết confidence của luật cao gấp khoảng 2.45 lần support nền của **other vegetables**. Đây là quan hệ đồng xuất hiện đáng chú ý để đưa vào thử nghiệm gợi ý mua kèm, nhưng chưa phải bằng chứng về tác động kinh doanh hay quan hệ nhân quả.

Luật **root vegetables $\Rightarrow$ other vegetables** có support count 466 và lift 2.2466. So với luật đứng đầu, luật này bao phủ nhiều giao dịch hơn nhưng có độ lệch khỏi mức nền thấp hơn. Do đó, luật có thể phù hợp hơn cho một thử nghiệm có phạm vi rộng, nhưng quyết định triển khai còn phụ thuộc biên lợi nhuận, vị trí kệ và phản ứng của khách hàng. Các luật liên quan đến **butter**, **curd** hoặc **domestic eggs** với **whole milk** có tính dễ diễn giải, song trực giác sản phẩm không thay thế việc kiểm thử thực tế.

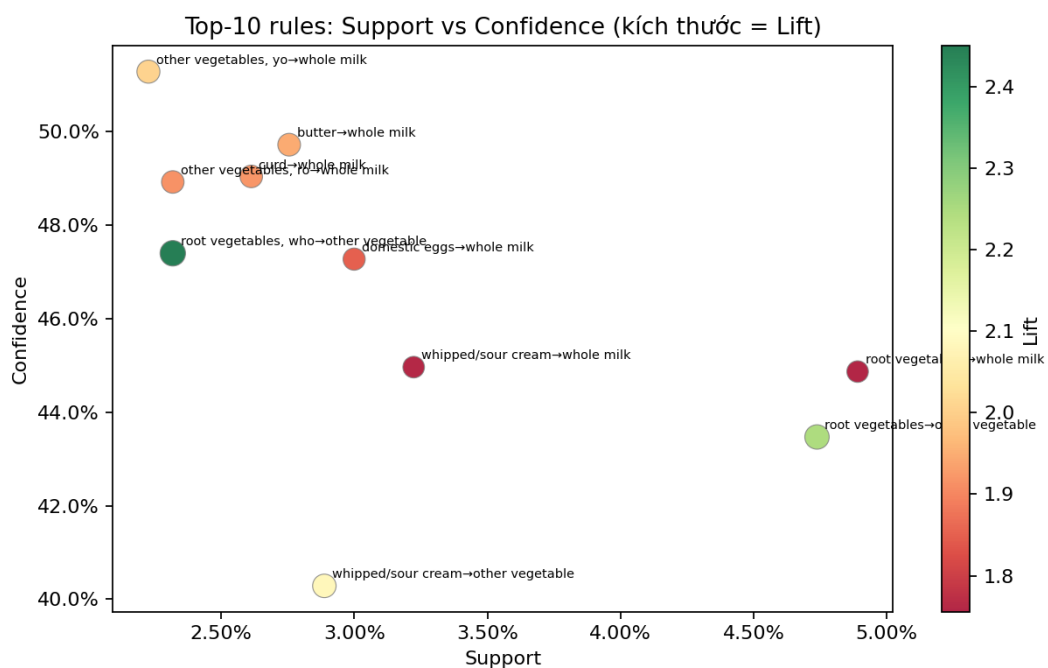
5.7 Trực quan hóa luật kết hợp

Để dễ phân tích hơn, notebook tạo ba biểu đồ trực quan hóa luật kết hợp: heatmap theo lift, bubble chart giữa support và confidence, và biểu đồ cột so sánh lift.



Hình 11: Heatmap lift của top 10 luật kết hợp

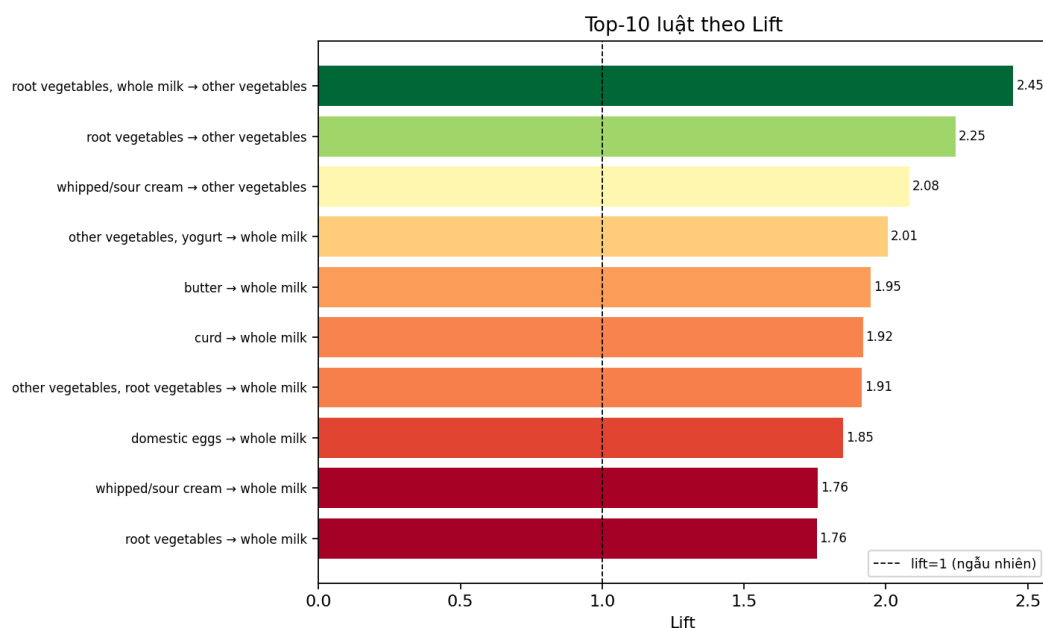
Hình 11 cho thấy các cặp antecedent–consequent có lift cao nhất. Các ô có lift lớn là ứng viên tốt để phân tích sâu hơn, nhưng cần lưu ý rằng heatmap không thể hiện trực tiếp số giao dịch hỗ trợ. Vì vậy, một luật có màu nổi bật vẫn cần được kiểm tra thêm bằng support count.



Hình 12: Bubble chart: quan hệ giữa support, confidence và lift của top 10 luật

Hình 12 biểu diễn mỗi luật bằng một điểm, trong đó trục hoành là support, trục tung là confidence, còn kích thước và màu sắc thể hiện lift. Biểu đồ này giúp nhận biết

các luật cân bằng hơn giữa độ phổ biến và độ tin cậy. Những luật có lift cao nhưng support thấp nên được xem như ứng viên thử nghiệm nhỏ, thay vì triển khai rộng ngay lập tức.



Hình 13: So sánh lift của top 10 luật kết hợp

Hình 13 so sánh các luật theo lift. Đường tham chiếu $\text{lift} = 1$ biểu diễn mức kỳ vọng dưới giả thuyết độc lập theo chỉ số này. Tất cả 10 luật đều có lift lớn hơn 1, tức chúng đồng xuất hiện nhiều hơn mức nền tương ứng. Tuy nhiên, lift có thể nhảy với các item hiếm và không tự bảo đảm ý nghĩa thống kê hay giá trị kinh doanh [15]. Vì vậy, cần xem thêm support, confidence, leverage và support count.

5.8 Diễn giải kinh doanh và đề xuất hành động

Từ 10 luật đứng đầu, nhóm xây dựng một số **giả thuyết hành động** để kiểm thử. Các đề xuất trong Bảng 37 không nên được xem là quyết định triển khai trực tiếp, vì dữ liệu hiện tại không chứa giá, lợi nhuận, mùa vụ hoặc phản hồi sau can thiệp.

Bảng 37: Diễn giải một số luật tiêu biểu và đề xuất hành động

Luật kết hợp	Ý nghĩa	Mức ưu tiên thử nghiệm	Đề xuất hành động
<code>root vegetables, whole milk $\Rightarrow$ other vegetables</code>	Khách mua rau củ gốc và sữa có xu hướng mua thêm nhóm rau khác; lift đạt 2.45.	Cao	Gợi ý <code>other vegetables</code> khi khách đã có <code>root vegetables</code> và <code>whole milk</code> ; thử combo rau củ + sữa.
<code>root vegetables $\Rightarrow$ other vegetables</code>	Luật có support count 466, phù hợp hơn để áp dụng rộng.	Cao	Thử nghiệm đặt gần hoặc tạo gợi ý mua kèm trong khu vực rau củ.
<code>whipped/sour cream $\Rightarrow$ other vegetables</code>	Quan hệ mua kèm mạnh với lift 2.08, nhưng support ở mức vừa.	Trung bình	Kiểm thử khuyến mãi nhỏ hoặc gợi ý theo công thức nấu ăn.
<code>other vegetables, yogurt $\Rightarrow$ whole milk</code>	Confidence cao nhất trong top 10, đạt 51.29%.	Cao	Gợi ý <code>whole milk</code> khi khách mua cả <code>other vegetables</code> và <code>yogurt</code> .
<code>butter $\Rightarrow$ whole milk</code>	Quan hệ hợp lý về mặt tiêu dùng, lift 1.95 và confidence 49.72%.	Trung bình	A/B test combo bơ + sữa hoặc gợi ý mua kèm tại quầy thực phẩm lạnh.
<code>domestic eggs $\Rightarrow$ whole milk</code>	Cặp sản phẩm bữa sáng/thực phẩm hằng ngày, support count 295.	Trung bình	Thử nghiệm gợi ý theo bữa ăn hoặc vị trí gần khu thực phẩm thiết yếu.

Các luật liên quan đến `root vegetables`, `other vegetables`, `whole milk` và `yogurt` là nhóm dễ diễn giải nhất vì chúng phản ánh hành vi mua các sản phẩm thực phẩm thiết yếu trong cùng giỏ hàng. Với các luật có support tương đối cao, cửa hàng có thể áp dụng ở mức rộng như bố trí kệ hoặc gợi ý mặc định. Với các luật có lift cao nhưng support thấp hơn, hướng phù hợp hơn là kiểm thử bằng khuyến mãi nhỏ hoặc gợi ý có điều kiện.

Một điểm quan trọng là không nên chỉ chọn luật có lift cao nhất. Luật có lift cao có thể xuất hiện trong ít giao dịch, dẫn đến tác động kinh doanh hạn chế. Do đó, khi chọn luật để triển khai, nhóm đề xuất xem xét đồng thời ba yếu tố: lift lớn hơn 1, confidence đủ cao và support count đủ lớn. Ngoài ra, cần cân nhắc bối cảnh sản phẩm, mùa vụ, giá bán và khả năng bố trí thực tế trong cửa hàng.

5.9 Hạn chế của phần ứng dụng

Mặc dù kết quả luật kết hợp có thể hỗ trợ ra quyết định, phần ứng dụng vẫn có một số hạn chế:

- Dữ liệu chỉ cho biết sự hiện diện của item trong giao dịch; không có số lượng mua,

giá, lợi nhuận, thời gian, cửa hàng hoặc thông tin khách hàng.

- Luật kết hợp phản ánh đồng xuất hiện, không chứng minh quan hệ nhân quả. Chiều $X \Rightarrow Y$ cũng không có nghĩa X xảy ra trước Y .
- Việc thử nhiều cặp ngưỡng rồi chọn các luật nổi bật có thể tạo thiên lệch lựa chọn. Chương chưa sử dụng hold-out theo thời gian hoặc dữ liệu cửa hàng khác để kiểm tra độ ổn định.
- Lift có thể đánh giá cao một số quan hệ hiếm; do đó cần đọc cùng support count và, khi cần, sử dụng kiểm định hoặc khoảng tin cậy [15].
- Dữ liệu không cho phép kiểm soát các yếu tố gây nhiễu như mùa vụ, chương trình khuyến mãi hoặc vị trí trưng bày hiện tại.
- Phân tích mới dừng ở mức offline. Hiệu quả kinh doanh chỉ có thể được xác nhận bằng A/B test hoặc thử nghiệm có đối chứng với các chỉ số như tỉ lệ mua kèm, doanh thu tăng thêm và lợi nhuận.

5.10 Kết luận chương

Chương này đã minh họa cách áp dụng thuật toán khai thác frequent itemset vào bài toán phân tích giỏ hàng. Từ dữ liệu `groceries.txt`, nhóm tiền xử lý được 9,835 giao dịch với 169 item khác nhau, sau đó sử dụng cài đặt PrePost/optimized qua Julia CLI để khai thác frequent itemset. Với ngưỡng `minsup = 0.020` và `minconf = 0.40`, thuật toán sinh được 122 frequent itemset và 15 association rules.

Các luật nổi bật chủ yếu xoay quanh `root vegetables`, `other vegetables`, `whole milk`, `yogurt`, `butter` và `domestic eggs`. Chúng cung cấp các giả thuyết hợp lý cho bố trí sản phẩm, combo hoặc gợi ý mua kèm. Tuy nhiên, kết quả chỉ phản ánh mẫu đồng xuất hiện trong dữ liệu lịch sử. Trước khi triển khai, cần xem xét đồng thời support, confidence, lift, leverage, support count và tiến hành thử nghiệm có đối chứng để đo tác động kinh doanh.

Tài liệu tham khảo

- [1] R. Agrawal and R. Srikant, ?Fast Algorithms for Mining Association Rules in Large Databases,? *in Proceedings of the 20th International Conference on Very Large Data Bases* Morgan Kaufmann, 1994, **pages** 487–499.
- [2] J. Han, M. Kamber and J. Pei, *Data Mining: Concepts and Techniques*, 3 **edition**. Morgan Kaufmann, 2011.
- [3] M. J. Zaki, ?Scalable Algorithms for Association Mining,? *IEEE Transactions on Knowledge and Data Engineering*, **jourvol** 12, **number** 3, **pages** 372–390, 2000. DOI: [10.1109/69.846291](https://doi.org/10.1109/69.846291)
- [4] J. Han, J. Pei and Y. Yin, ?Mining Frequent Patterns without Candidate Generation,? *in Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data* ACM, 2000, **pages** 1–12. DOI: [10.1145/335191.335372](https://doi.org/10.1145/335191.335372)
- [5] Z.-H. Deng, Z.-H. Wang and J.-J. Jiang, ?A New Algorithm for Fast Mining Frequent Itemsets Using N-lists,? *Science China Information Sciences*, **jourvol** 55, **number** 9, **pages** 2008–2030, 2012. DOI: [10.1007/s11432-012-4638-z](https://doi.org/10.1007/s11432-012-4638-z)
- [6] J. M. Luna, P. Fournier-Viger and S. Ventura, ?Frequent Itemset Mining: A 25 Years Review,? *WIREs Data Mining and Knowledge Discovery*, **jourvol** 9, **number** 6, e1329, 2019. DOI: [10.1002/widm.1329](https://doi.org/10.1002/widm.1329)
- [7] Z.-H. Deng and S.-H. Lv, ?PrePost+: An Efficient N-lists-based Algorithm for Mining Frequent Itemsets via Children–Parent Equivalence Pruning,? *Expert Systems with Applications*, **jourvol** 42, **number** 13, **pages** 5424–5432, 2015. DOI: [10.1016/j.eswa.2015.03.004](https://doi.org/10.1016/j.eswa.2015.03.004)
- [8] P. Fournier-Viger, A. Gomariz, T. Gueniche, A. Soltani, C.-W. Wu and V. S. Tseng, ?SPMF: A Java Open-Source Pattern Mining Library,? *Journal of Machine Learning Research*, **jourvol** 15, **pages** 3389–3393, 2014. url: <https://www.jmlr.org/papers/v15/fournierviger14a.html>
- [9] P. Fournier-Viger, *SPMF: An Open-Source Data Mining Library*, <https://www.philippe-fournier-viger.com/spmf/>, Accessed: 2026-06-22.
- [10] Frequent Itemset Mining Implementations Repository, *FIMI Repository: Datasets*, <https://fimi.uantwerpen.be/data/>, Accessed: 2026-06-22.
- [11] B. Goethals and M. J. Zaki, ?Advances in Frequent Itemset Mining Implementations: Report on FIMI’03,? *in ACM SIGKDD Explorations Newsletter* **volume** 6, 2004, **pages** 109–117. DOI: [10.1145/1007730.1007744](https://doi.org/10.1145/1007730.1007744)

- [12] J. Bezanson, A. Edelman, S. Karpinski **and** V. B. Shah, ?Julia: A Fresh Approach to Numerical Computing,? *SIAM Review*, **jourvol** 59, **number** 1, **pages** 65–98, 2017. DOI: [10.1137/141000671](https://doi.org/10.1137/141000671)
- [13] M. Hahsler, B. Grün **and** K. Hornik, ?arules: A Computational Environment for Mining Association Rules and Frequent Item Sets,? *Journal of Statistical Software*, **jourvol** 14, **number** 15, **pages** 1–25, 2005. DOI: [10.18637/jss.v014.i15](https://doi.org/10.18637/jss.v014.i15)
- [14] S. Brin, R. Motwani, J. D. Ullman **and** S. Tsur, ?Dynamic Itemset Counting and Implication Rules for Market Basket Data,? **in** *Proceedings of the 1997 ACM SIGMOD International Conference on Management of Data* ACM, 1997, **pages** 255–264. DOI: [10.1145/253260.253325](https://doi.org/10.1145/253260.253325)
- [15] M. Hahsler **and** K. Hornik, ?New Probabilistic Interest Measures for Association Rules,? *Intelligent Data Analysis*, **jourvol** 13, **number** 4, **pages** 615–633, 2009. DOI: [10.3233/IDA-2009-0387](https://doi.org/10.3233/IDA-2009-0387)

A Phụ lục

A.1 Mã nguồn và notebook thực nghiệm

Dự án gồm ba notebook chính được đặt trong thư mục `notebooks/`:

- `manual_example.ipynb`: minh họa từng bước hoạt động của thuật toán PrePost trên ví dụ nhỏ, bao gồm xây dựng PPC-tree, gán mã `pre`, mã `post`, tạo N-list và khai thác frequent itemsets.
- `experiment_analysis.ipynb`: phân tích kết quả thực nghiệm ở Chương 4, đọc các file kết quả trong thư mục `results/`, tổng hợp thời gian chạy, số lượng frequent itemsets và trực quan hóa kết quả benchmark.
- `market_basket_analysis.ipynb`: minh họa ứng dụng PrePost vào bài toán Market Basket Analysis trên dữ liệu groceries. Notebook này thực hiện đọc dữ liệu, tiền xử lý giao dịch, khai thác frequent itemsets, sinh luật kết hợp, tính các chỉ số đánh giá và trực quan hóa kết quả.

Các hình ảnh sử dụng trong Chương 4 và Chương 5 được xuất tự động từ notebook `experiment_analysis.ipynb` và `demo.ipynb` và lưu trong thư mục `figures/`.

A.2 Môi trường thực nghiệm

Các thí nghiệm trong báo cáo được thực hiện bằng ngôn ngữ Julia cho phần cài đặt thuật toán PrePost và Python cho phần phân tích dữ liệu, sinh luật kết hợp và trực quan hóa kết quả. Bộ công cụ SPMF được sử dụng làm chương trình tham chiếu để kiểm tra độ đúng của frequent itemsets, không được sử dụng trong quá trình cài đặt thuật toán PrePost.

Các yêu cầu hệ thống chính gồm:

- Julia phiên bản từ 1.9 trở lên.
- Python phiên bản từ 3.9 trở lên.
- Jupyter Notebook hoặc JupyterLab.
- Java phiên bản từ 8 trở lên nếu cần chạy SPMF tham chiếu.
- Các thư viện Python như `pandas`, `numpy`, `matplotlib`, `seaborn`, `mlxtend`.

A.3 Cấu trúc thư mục

```
PrePost/
|
|-- README.md
|-- Project.toml
|-- Manifest.toml
|
|-- src/
|   |-- PrePostFIM.jl
|   |-- cli.jl
|   |-- algorithm/
|   |-- structures/
|   |-- utils/
|
|-- tests/
|
|-- data/
|   |-- toy/
|   |-- benchmark/
|   |-- subsets/
|   |-- synthetic/
|   |-- application/
|
|-- notebooks/
|   |-- manual_example.ipynb
|   |-- experiment_analysis.ipynb
|   |-- market_basket_analysis.ipynb
|
|-- outputs/
|-- logs/
|
|-- docs/
|   |-- Report.tex
|   |-- references.bib
|   |-- chapters/
|   |-- figures/
```

A.4 Hướng dẫn tái lập kết quả

Quy trình chạy khuyến nghị như sau:

1. Cài đặt dữ liệu và môi trường Julia.
2. Chạy bộ kiểm thử để kiểm tra project.
3. Chạy thuật toán hoặc benchmark vì notebook cần đọc kết quả từ thư mục `outputs/`
4. Mở Jupyter Notebook hoặc JupyterLab từ thư mục gốc project.
5. Chạy từng notebook theo thứ tự từ trên xuống dưới, trừ `manual_example.ipynb` có thể chạy sau khi có datasets toy.

Trên Windows, có thể chạy:

```
.\cmd\setup_datasets.ps1  
.\cmd\setup_julia_env.ps1  
.\cmd\run_all_experiments.ps1 -Spmf
```

Trên Linux hoặc macOS, có thể chạy:

```
bash scripts/setup_datasets.sh  
bash scripts/setup_julia_env.sh  
bash scripts/run_all_experiments.sh -Spmf
```

A.5 Ghi chú về kết quả sinh tự động

Thư mục `outputs/`, chứa các file được sinh tự động trong quá trình chạy thuật toán, benchmark. Khi cần tái hiện toàn bộ kết quả, nên chạy lại các script từ đầu thay vì chỉnh sửa thủ công các file trong những thư mục này.